


# AULA INAUGURAL
# PROGRAMAÇÃO PARALELA

Prof. Rodrigo Gonçalves

# Assuntos importantes

## Cronograma das aulas

- → Ementa, Objetivo, cronograma, avaliações e bibliografia.
- → Aulas.
- → Exercícios.
- → Comunicação personalizada.
- → ChatGPT como utilizar.
- → Google Classroom.
- → Introdução à Programação Paralela
- → Exercícios.



# Ementa

## Ementa

→.

# Ementa

## Objetivo Geral

→ .

# Ementa

## Objetivo Específico

→ .

# Ementa

## Cronograma

→ .

# Ementa

## Avaliação

→ .

# Ementa

## Bibliografia

→ .

# Ementa

## Aulas

→ Início : 8h15
→ Intervalo: 9h30 até 9h45

→ Fim: 12h15.



**É Proibido gravar aula em áudio e/ou vídeo, assim como distribuir este material a terceiros. Sujeito ás penalidades da lei de direitos autorais**

- ✓ Curriculum do professor.
- ✓ Hobby → resolver equações matemáticas no café da manhã.



www.linkedin.com/in/rodrigouzz



Stanford
University




Centro Universitário
A Teoria e a Prática Juntas

# Por que Paralelismo?

# Por que Eficiência?

Nosso curso falará desses dois aspectos



## Qual a definição de paralelismo quando pensamos em computação paralela?

**Um computador paralelo é uma coleção de elementos de processamento que colaboram para resolverem problemas rapidamente**




Estamos preocupados com performance e Eficiência

Utilizaremos múltiplos processadores para isso

## Algumas vezes eficiência é muito mais importante que paralelismo

**Para entendermos esses conceitos, vamos ao um pequeno jogo.**




**Tem algum aluno que pode suportar algum tipo de pressão?**

**Escolher os times!**

## Algumas vezes eficiência é muito mais importante que paralelismo

**Para entendermos esses conceitos, vamos ao um pequeno jogo.**

**Tem algum aluno que pode suportar algum tipo de pressão?**

**1º um aluno, como processador. Pacote 1, 10 números. SOMA**

**Cronometrar a soma**

Prof. Rodrigo Gonçalves IESB

Podemos praticar bastante por alguns dias essa soma?

De quais outras formas podemos realizar essa conta mais rápido??

Quero sugestões!

**Como podemos fazer melhor e mais rápido do o primeiro Aluno(a)?**

Vamos agora fazer a conta com 2 alunos. 1º aluno na frente da sala, outro aluno no fundo da sala. Vocês dois, não podem se comunicar verbalmente.
Apenas por papel. Cada aluno terá 5 números. Quando tiverem o valor total, sinalizem.

# Vamos analisar os resultados

**1º Aluno(a) fez o cálculo em =**

**1 recurso, um  investimento, vamos supor que 1 milhão de reais.**

**2 Alunos(as) fizeram o cálculo em =**

**2 recursos, um  investimento dobrado, vamos supor que 2 milhões de reais. Não teríamos que diminuir o dobro do tempo??**

**Qual a porcentagem de quem fez melhor?**

**O que aconteceu aqui? Next slide →**

**Se assumirmos que todos os alunos sabem matemática aproximadamente com a mesma rapidez.**

# Qual é a opinião de vocês de como fazer melhor que isso?

**Teríamos que melhorar a comunicação.**

Prof. Rodrigo Gonçalves IESB

# Speedup ou  aceleração ou velocidade

**A maior motivação de usar processamento paralelo é alcançar uma velocidade ou acelaração (speedup)**

Para um problema:

$$\text{Speedup (using P processors)} = \frac{\text{tempo de execução usando 1 processador}}{\text{tempo de execução usando P processadores}}$$

# Speedup

→ **Speedup**: é uma métrica que quantifica quanto mais rápido um programa roda com múltiplos processadores em comparação com a execução sequencial

$$\textbf{Speedup} = \frac{T_1}{T_p}$$

# Speedup

→ **Exemplo**

T1 = 100 segundos com 1 processador

T4 = 30 segundos com 4 processadores. → 4 processadores

$$\textbf{Speedup} = \frac{100}{30} \approx 3,33$$

Isso significa que com 4 processadores, o programa é 3,33 vezes mais rápido do que com 1.

## Vamos voltar a comunicação.

→ **Se duas pessoas** é difícil melhorar a comunicação, vamos tentar 4 alunos agora.

**1º Os 4 alunos podem ficar um do lado do outro.**

**2º dessa vez, os quatros alunos podem se comunicar verbalmente. (better communications)**

## Vamos voltar a comunicação.

→ **Se duas pessoas** é difícil melhorar a comunicação, vamos tentar 4 alunos agora.

**1º Os 4 alunos podem ficar um do lado do outro.**

**2º dessa vez, os quatros alunos podem se comunicar verbalmente. (better communications)**

# Vamos calcular o novo speedup

$$\text{Speedup} = \frac{T_1}{T_p}$$

**O que podemos concluir disso, pessoal?**

→ Número de processadores, chamamos de Workers. 4 processadores, temos 4 Workers.

Alguém ficou com números grandes, maiores para calcular? Ou seja, um worker teve uma carga mais pesada, outro worker uma carga mais leve. Será que estamos utilizando bem nossos Workers? A distribuição entre os Workers foram iguais? Os que ficaram com números mais fáceis, ficaram esperando pelo o worker com o cálculo mais difícil.

# Como podemos melhorar ainda mais? Estratégias?

Prof. Rodrigo Gonçalves IESB

## Como podemos melhorar ainda mais?

**Agora faremos com a classe toda.**

**Discutam entre vocês sobre estratégias, antes de começarmos.**

**O resultado final deve vir até mim.**

# O que foi bom ou ruim dessa última experiência?



## Observações do problema da sala, com a dinâmica da sala toda.

→ O problema que passei a vocês tem uma quantidade significativa de comunicação quando se comparado com a computação, certo.

**→ Os custos de comunicação podem dominar a computação paralela SEVERAMENTE, limitando o speedup (aceleração)**

**→ Guardem esse aprendizado, porque na realidade o que importa no paralelismo, é comunicação, e movimentação de elementos.**

# Eficiência

→ O que é eficiência?

→ Rápido != Eficiência

→ Só porque seus programas rodam muito mais rápidos em um computador paralelo, que significa que ele está usando o hardware eficientemente.

→ Pensamento: um computador com 10 processadores é considerado muito bom, ter um speedup 2x? Se comparado com um computador de 1 processador?



# Eficiência E Velocidade (speedup)

→ Ideal é conseguir os 2



# Programação em C/C++ / Python

→ Como está a programação de vocês nessas linguagens?



# Por quê Paralelismo?

## Some historical context: why avoid parallel processing?

- Single-threaded CPU performance doubling ~ every 18 months
- Implication: working to parallelize your code was often not worth the time
  - Software developer does nothing, code gets faster next year. Woot!

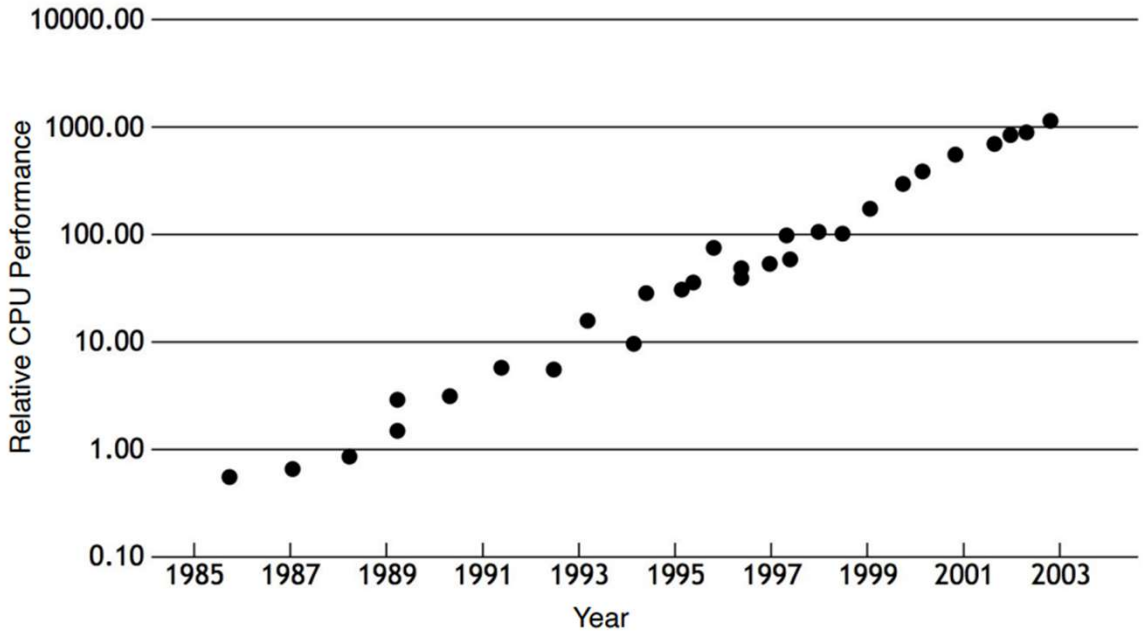



Image credit: Olukutun and Hammond, ACM Queue 2005

# Por quê Paralelismo?

→ Até 17 anos atrás, havia 2 razões significativas para melhorar a performance de processadores.

→ 1. Explorar instruções a nível de paralelismo

→ 2. Aumentar CPU clock frequency



# Por quê Paralelismo?

→ Por quê os processadores aumentam logisticamente ou quase exponencialmente a performance?

**→Porque os transistores estão cada vez menores!!!**




→ Com isso, melhorou a frequência de comunicação

→ Mais transistores podem estar no mesmo chip.

→ E arquitetos descobriram formas inteligentes de usar esses transistores para traduzi-los em performance.

→ Vamos voltar a base! Pergunto a vocês.

→ Olhe para o colega do lado, e respondam a si mesmos ...

# O que é um programa de computador? Da perspectiva do computador?




Prof. Rodrigo Gonçalves IESB

## Programa escrito em C

```
int main(int argc, char** argv) {

    int x = 1;

    for (int i=0; i<10; i++) {
       x = x + x;
    }

    printf("%d\n", x);

    return 0;
}
```



## Mas da perspectiva do computador, uma lista de instruções!

## assembly

# A program is just a list of processor instructions!

```
int main(int argc, char** argv) {

  int x = 1;

  for (int i=0; i<10; i++) {
    x = x + x;
  }

  printf("%d\n", x);

  return 0;
}
```




## → Instruções X86

```
_main:
100000f10:     pushq     %rbp
100000f11:     movq %rsp, %rbp
100000f14:     subq $32, %rsp
100000f18:     movl $0, -4(%rbp)
100000f1f:     movl %edi, -8(%rbp)
100000f22:     movq %rsi, -16(%rbp)
100000f26:     movl $1, -20(%rbp)
100000f2d:     movl $0, -24(%rbp)
100000f34:     cmpl $10, -24(%rbp)
100000f38:     jge  23 <_main+0x45>
100000f3e:     movl -20(%rbp), %eax
100000f41:     addl -20(%rbp), %eax
100000f44:     movl %eax, -20(%rbp)
100000f47:     movl -24(%rbp), %eax
100000f4a:     addl $1, %eax
100000f4d:     movl %eax, -24(%rbp)
100000f50:     jmp  -33 <_main+0x24>
100000f55:     leaq 58(%rip), %rdi
100000f5c:     movl -20(%rbp), %esi
100000f5f:     movb $0, %al
100000f61:     callq     14
100000f66:     xorl %esi, %esi
100000f68:     movl %eax, -28(%rbp)
100000f6b:     movl %esi, %eax
100000f6d:     addq $32, %rsp
```

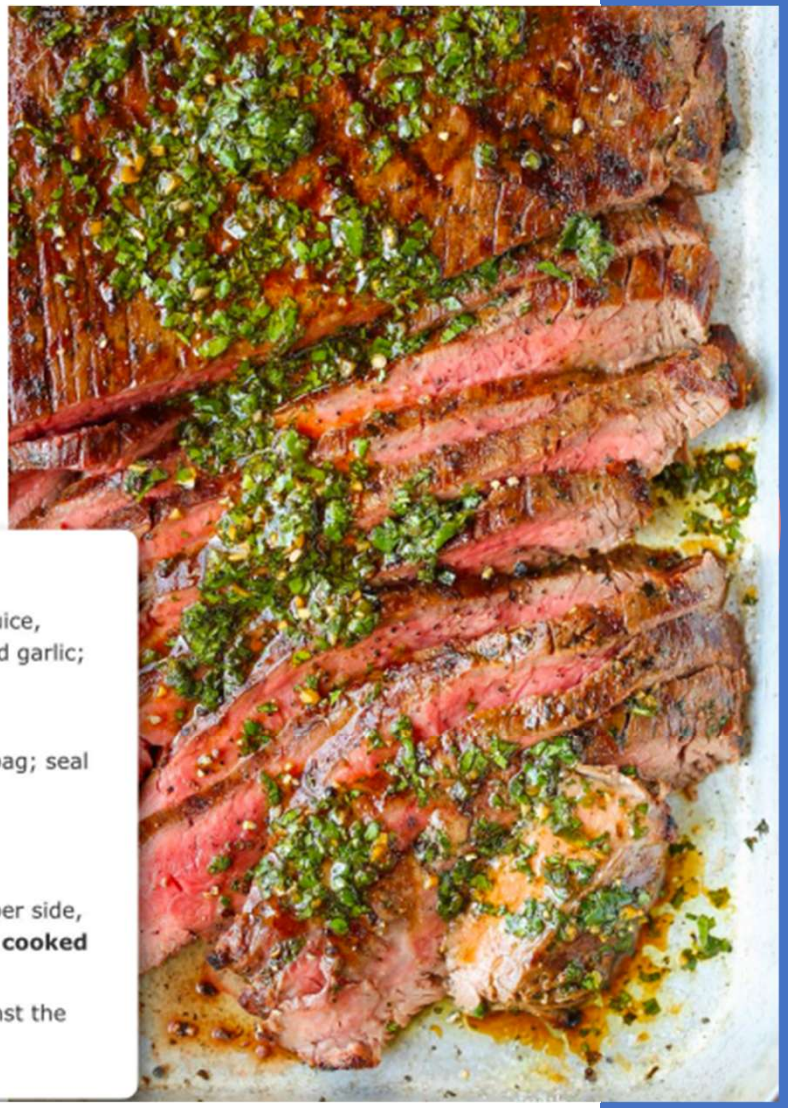
## O que são instruções?

# Kind of like the instructions in a recipe for your favorite meals

## Mmm, carne asada

**Instructions**

1. In a large mixing bowl combine orange juice, olive oil, cilantro, lime juice, lemon juice, white wine vinegar, cumin, salt and pepper, jalapeno, and garlic; whisk until well combined.
2. Reserve ⅓ cup of the marinade; cover the rest and refrigerate.
3. Combine remaining marinade and steak in a large resealable freezer bag; seal and refrigerate for at least 2 hours, or overnight.
4. Preheat grill to HIGH heat.
5. Remove steak from marinade and lightly pat dry with paper towels.
6. Add steak to the preheated grill and cook for another 6 to 8 minutes per side, or until desired doneness. **Note that flank steak tastes best when cooked to rare or medium rare because it's a lean cut of steak.**
7. Remove from heat and let rest for 10 minutes. Thinly slice steak against the grain, garnish with reserved cilantro mixture, and serve.



## E o que os processadores fazem?

# What does a processor do?



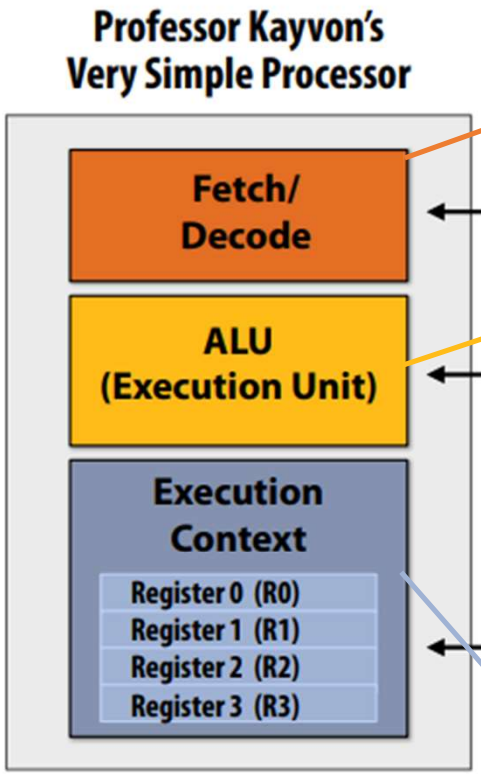






IESB
Centro Universitário
A Teoria e a Prática Juntas

# A processor executes instructions




Controle. Determina quais instruções serão as próximas. De onde está e para onde vai.

Determine what instruction to run next

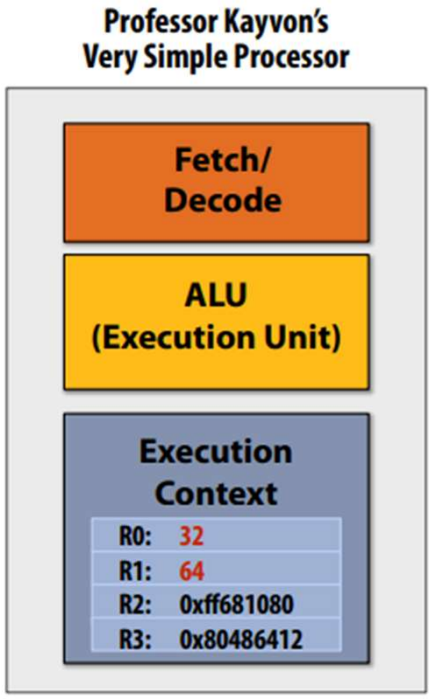
Responsável pela aritmética, movimento

Execution unit: performs the operation described by an instruction, which may modify values in the processor's registers or the computer's memory

Registers: maintain program state: store value of variables used as inputs and outputs to operations

Estado (state). Executor de contexto. Contém os registradores, onde os valores são inseridos

# Exemplo de instruções: adição de 2 números (add).




**Passo 1:**
**Processador pega o próxima instrução do programa em memória**

**add R0 ← R0, R1**

***"por favor, adicione o conteúdo do registro R0 ao conteúdo do registro R1 e coloque o resultado da adição no registro R0"***

***Passo 2:***
***Obter entradas dos registradores.***
***Conteúdo da entrada R0 na unidade de execução :*** 32
***Conteúdo da entrada R1 na unidade de execução:*** 64

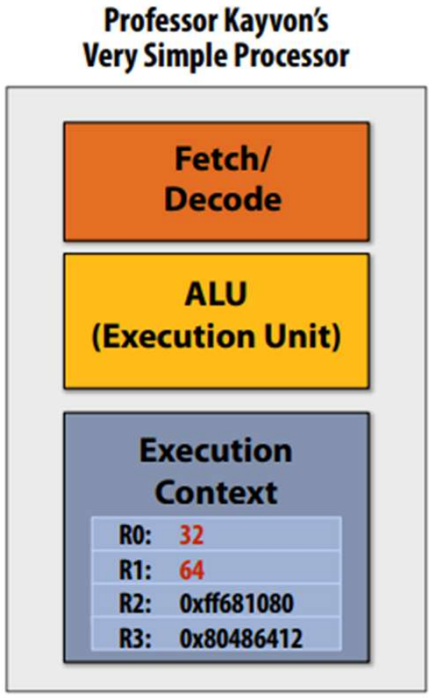
***Passo 3:***

***Executar a operação de adição:***

***A unidade de execução realiza a aritmética (soma), o resultado é:*** 96

***Continua ...***

# Exemplo de instruções: adição de 2 números (add), continuação...




**Passo 1:**
**Processador pega o próxima instrução do programa em memória**

**add R0 ← R0, R1**

***"por favor, adicione o conteúdo do registro R0 ao conteúdo do registro R1 e coloque o resultado da adição no registro R0"***

***Passo 2:***
***Obter entradas dos registradores.***
***Conteúdo da entrada R0 na unidade de execução :*** 32
***Conteúdo da entrada R1 na unidade de execução:*** 64

***Passo 3:***

***Executar a operação de adição:***

***A unidade de execução realiza a aritmética (soma), o resultado é:*** 96

***passo 4:***

***Grave o resultado*** 96 ***no registro R0***

# Então, voltando a nossa pergunta inicial.

→ O que é um programa de computador? ( da perspectiva do processador)

*Uma lista de instruções para executar!*

→ O que é uma instrução?

*Descreve uma operação que deve ser executada por um processador.*
*Executando uma instrução, modifica o estado do computador.*
*São comandos!*

→ O que eu quero dizer quando falo sobre o estado de um computador?

*Valores ou dados de um programa que são armazenados nos registradores de um processador ou memória. (muda seu estado atual).*

# Vamos considerar uma parte de um código muito simples.

**a = x*x + y*y + z*z**

**Consider the following five instruction program:**

*Assume register R0 = x, R1 = y, R2 = z*

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

*R3 now stores value of program variable 'a'*

**Este programa tem 5 instruções, então, para resolvermos esse problema, o processor utiliza 5 clocks para execução certo?**

**Podemos melhorar isso?**

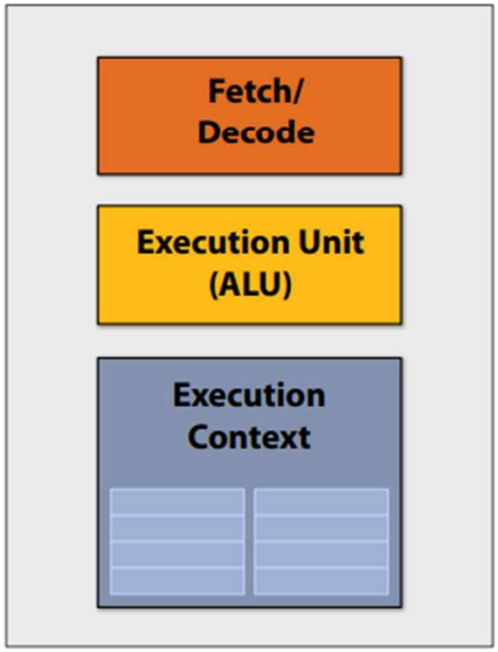


Assim como em uma recita, por exemplo carne assada, devemos ter uma ordem das instruções, senão não conseguiremos atingir o objetivo final.



# Execute program

## My very simple processor: executes one instruction per clock

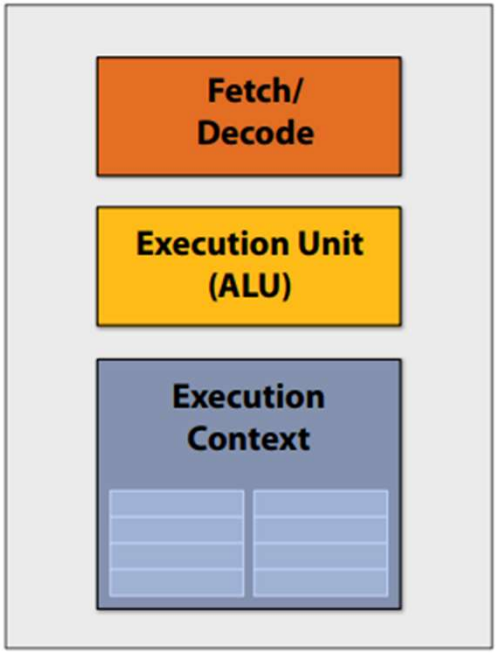



```
ld   r0, addr[r1]
mul  r1, r0, r0
mul  r1, r1, r0
...
...
...
...
...
...
st   addr[r2], r0
```



# Execute program

## My very simple processor: executes one instruction per clock

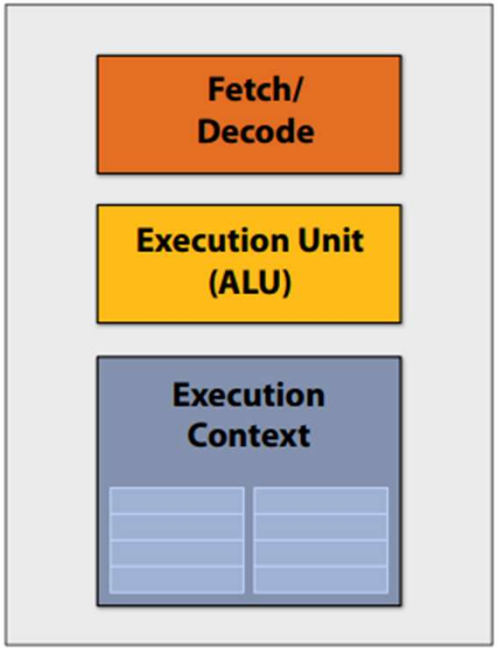



```
ld   r0, addr[r1]
mul  r1, r0, r0
mul  r1, r1, r0
...
...
...
...
...
...
st   addr[r2], r0
```



# Execute program

## My very simple processor: executes one instruction per clock

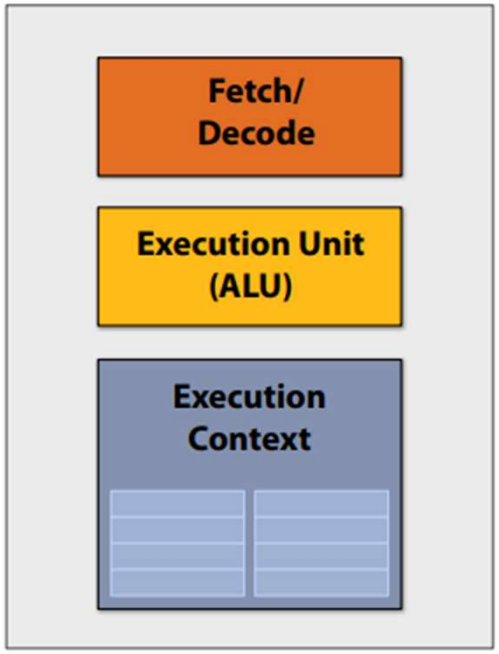



```
ld   r0, addr[r1]
mul  r1, r0, r0
mul  r1, r1, r0
...
...
...
...
...
...
st   addr[r2], r0
```



# Execute program

## My very simple processor: executes one instruction per clock




```
ld   r0, addr[r1]
mul  r1, r0, r0
mul  r1, r1, r0
...
...
...
...
...
...
st   addr[r2], r0
```



# Vamos considerar uma parte de um código muito simples.

**a = x*x + y*y + z*z**

```
1. mul R0, R0, R0     // x*x
2. mul R1, R1, R1     // y*y
3. mul R2, R2, R2     // z*z
4. add R0, R0, R1     // x*x + y*y
5. add R3, R0, R2     // (x*x + y*y) + z*z
```



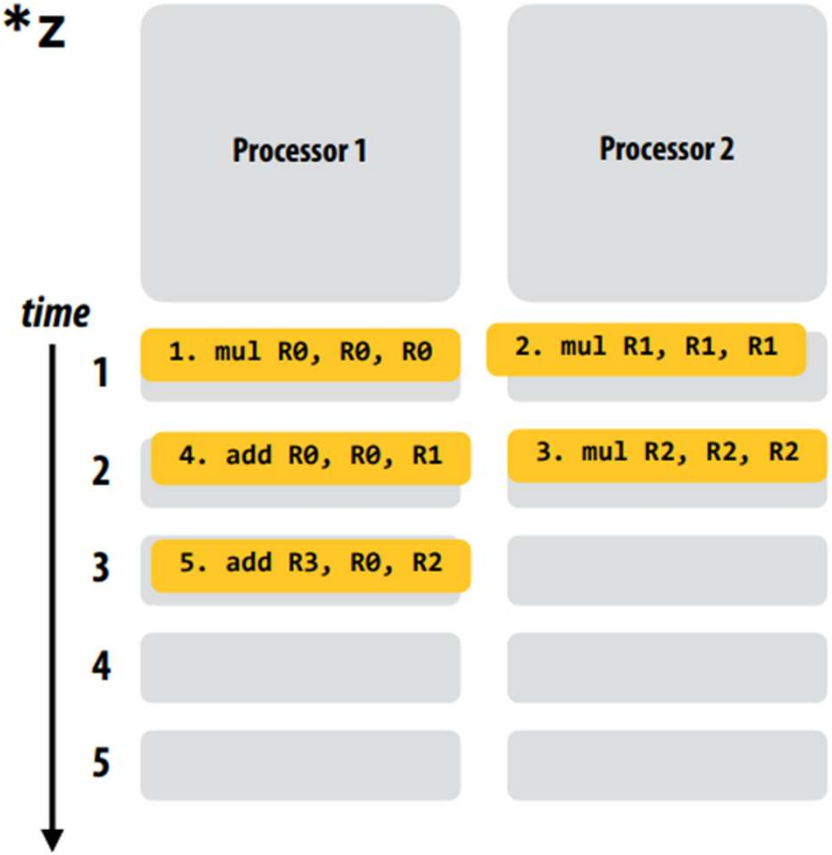
## E se colocarmos duas instruções para serem executadas ao mesmo tempo? Quantos ciclos temos aqui?

$$a = x*x + y*y + z*z$$

*Assume register*
*R0 = x, R1 = y, R2 = z*

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

*R3 now stores value of program variable 'a'*



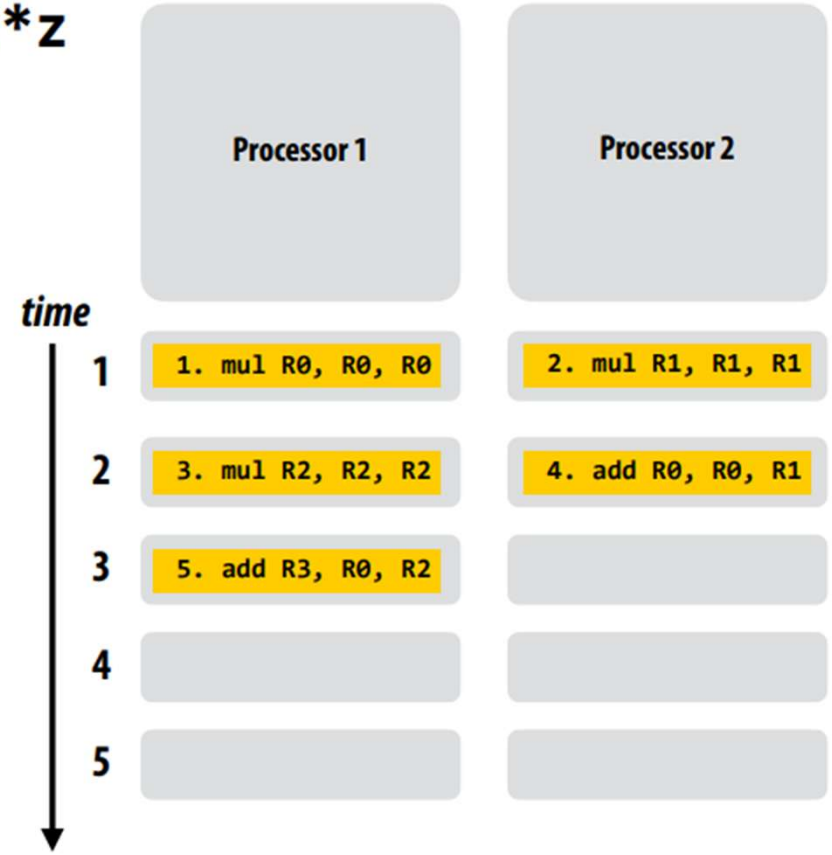
# E se colocarmos duas instruções para serem executadas ao mesmo tempo? Qtos Ciclos?

$$a = x*x + y*y + z*z$$

*Assume register*
*R0 = x, R1 = y, R2 = z*

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

*R3 now stores value of program variable 'a'*




Prof. Rodrigo Gonçalves IESB

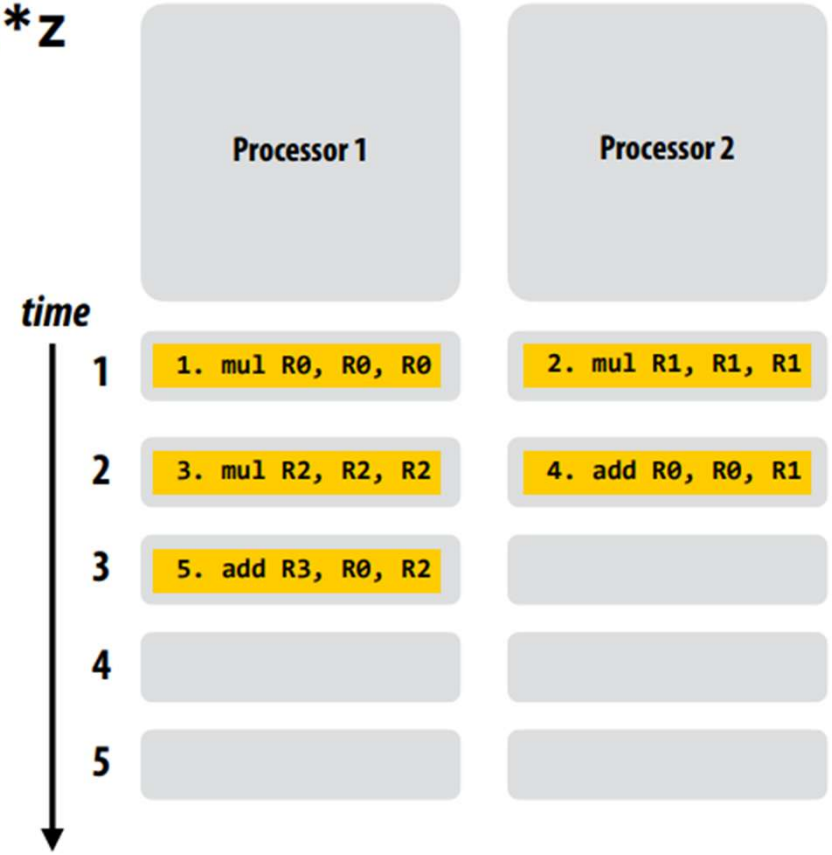
# E se colocarmos duas instruções para serem executadas ao mesmo tempo? Qtos Ciclos?

$$a = x*x + y*y + z*z$$

*Assume register*
*R0 = x, R1 = y, R2 = z*

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

*R3 now stores value of program variable 'a'*




Prof. Rodrigo Gonçalves IESB

## Como fazer que  com as instruções respeitem a ordem em que o programa foi escrito?

## Como fazer melhor?



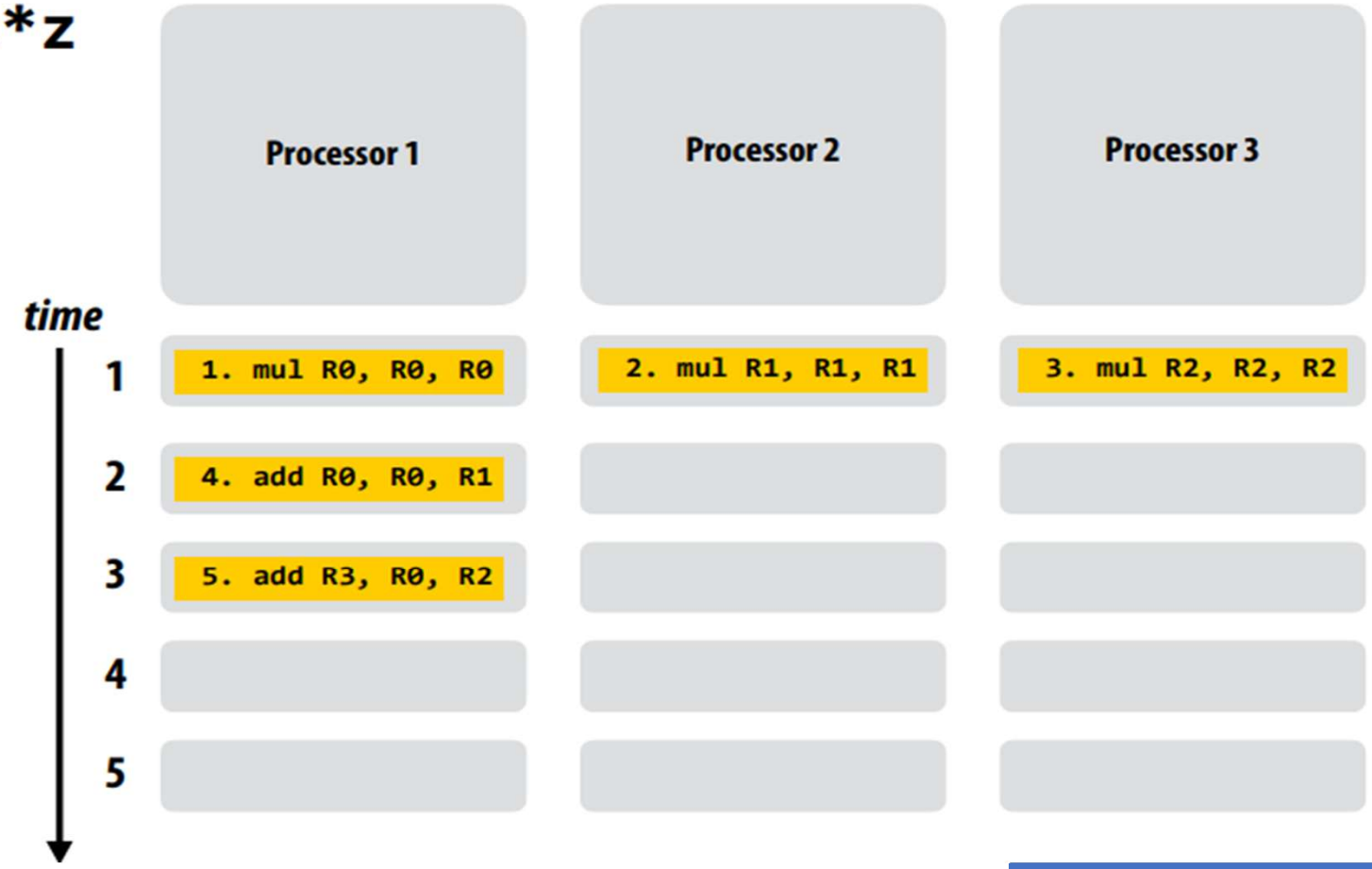
# E se colocarmos 3 processadores? Qtos Ciclos? Temos dependências!

$$a = x*x + y*y + z*z$$

*Assume register*
*R0 = x, R1 = y, R2 = z*

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

*R3 now stores value of program variable 'a'*



## Nível de instruções de Paralelismo - Exemplo

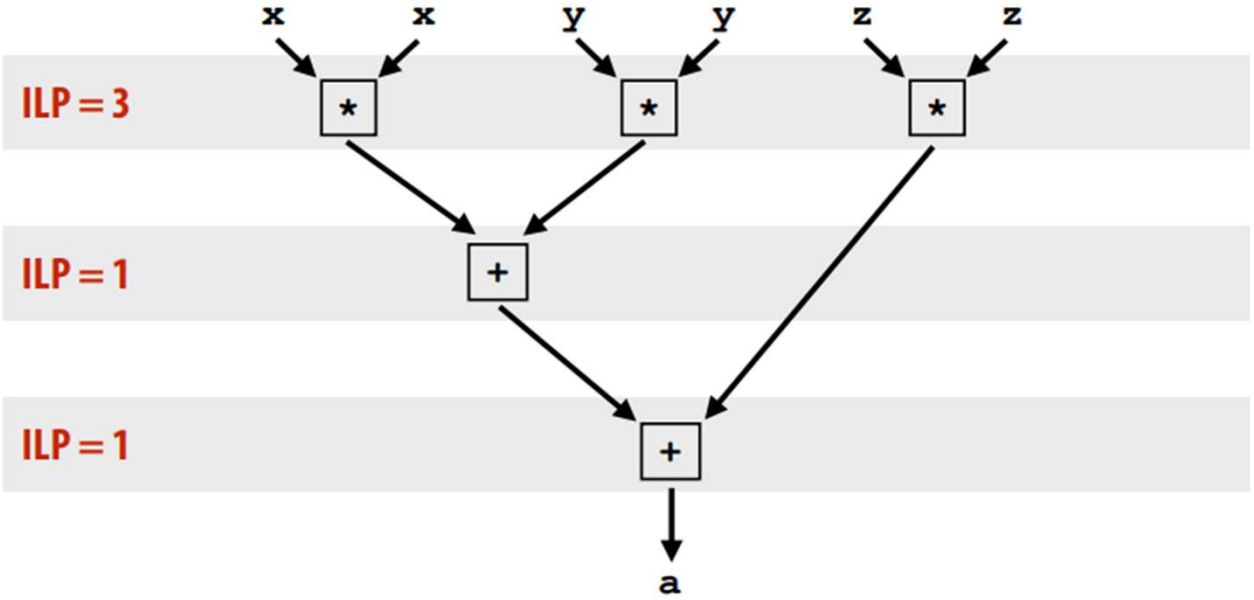
ILP: como executar múltiplas instruções em paralelo mesmo em um único núcleo?

# Instruction level parallelism (ILP) example

- ILP = 3

```
a = x*x + y*y + z*z
```

**Dependência**




**Será que mais transistores implicam em mais performance?**

**Nope. Porque temos dependências!**

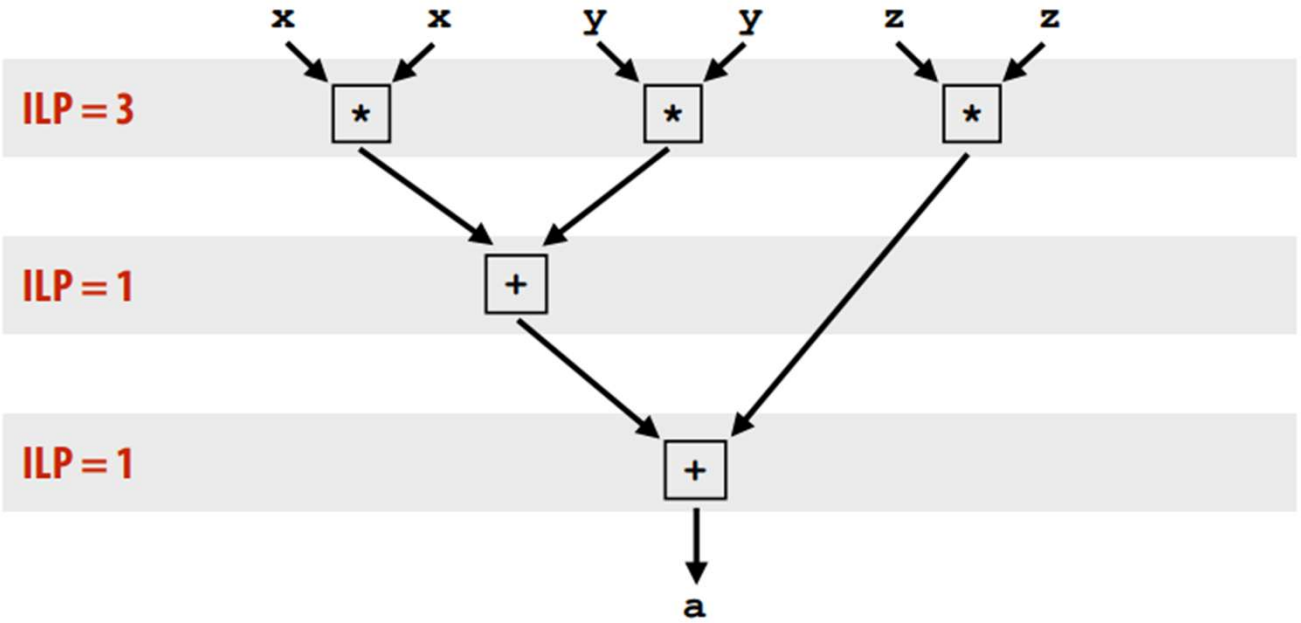
## Nível de instruções de Paralelismo - Exemplo

# Instruction level parallelism (ILP) example

- ILP = 3

```
a = x*x + y*y + z*z
```

**Dependência**



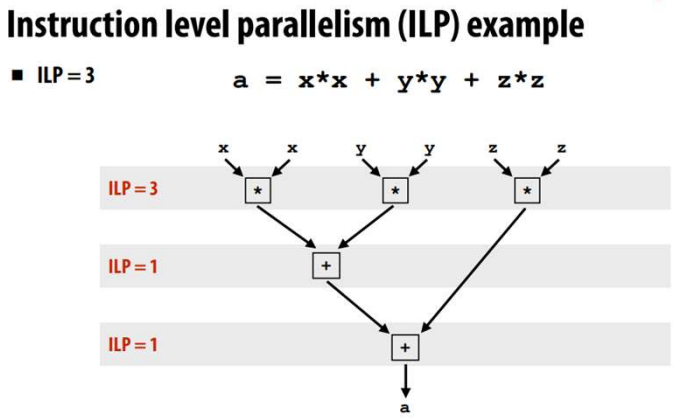

**Será que mais transistores implicam em mais performance?**

**Nope. Porque temos dependências!**

**Legal, já que temos dependências, como os computadores com processadores modernos executam instruções?**

**R. Superscalar Execution ou Processadores Superescalares**

**Ideia:** o processador automaticamente encontra instruções independentes em uma sequencia e as executam em paralelo em multiplas unidades de execuções (núcleo)




**Continua..**

## ...Continuação

**Consider the following five instruction program:**

*Assume register R0 = x, R1 = y, R2 = z*

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

*R3 now stores value of program variable 'a'*

**No exemplo que estamos trabalhando:** as instruções 1,2 e 3 podem ser executadas em paralelo sem impactar o programa (o processador superescalar que determina se as instruções são independentes). No entanto a instrução 4, deve ser executada após as instruções 1 e 2.

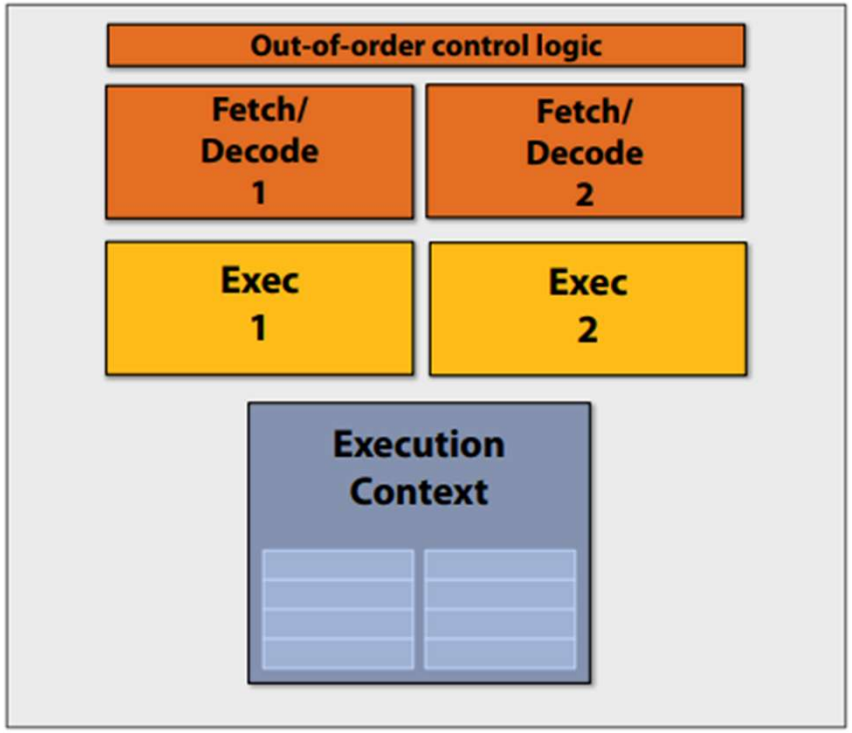
**E a instrução 5 deve ser executada após a instrução 4.**

Esse processador superescalar (2 núcleos) pode decodificar 2 instruções por clock.

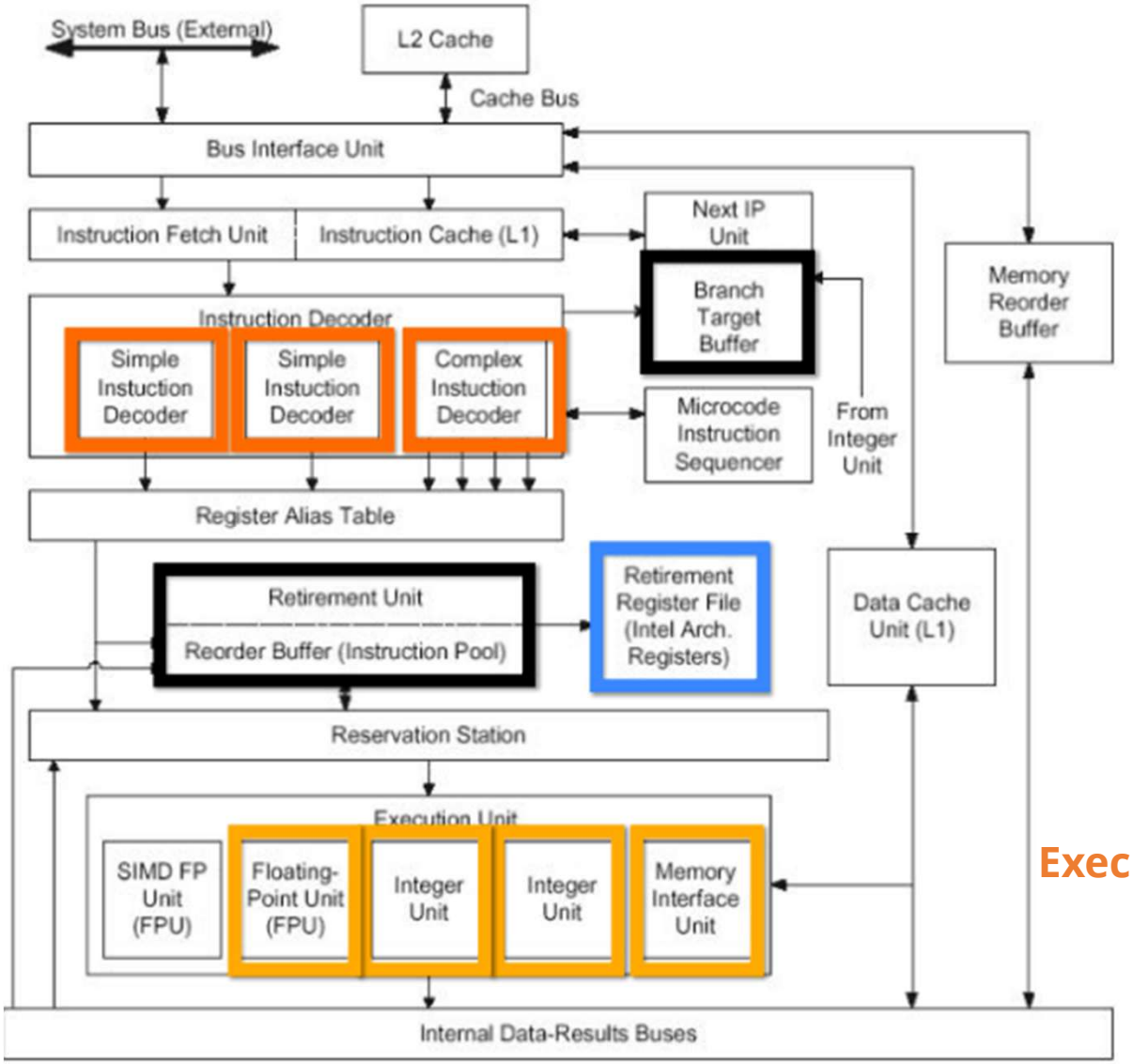
# Superscalar processor

**This processor can decode and execute up to two instructions per clock**

Reordena instruções independentes para serem executadas



# Arquitetura antiga do processador Pentium 4




IESB
Centro Universitário
A Teoria e a Prática Juntas

**Executa 3/4 instruções ao mesmo tempo.**

**Prof. Rodrigo Gonçalves IESB**

# Outro exemplo, um pouco mais complexo

## Instruções de 1 processador

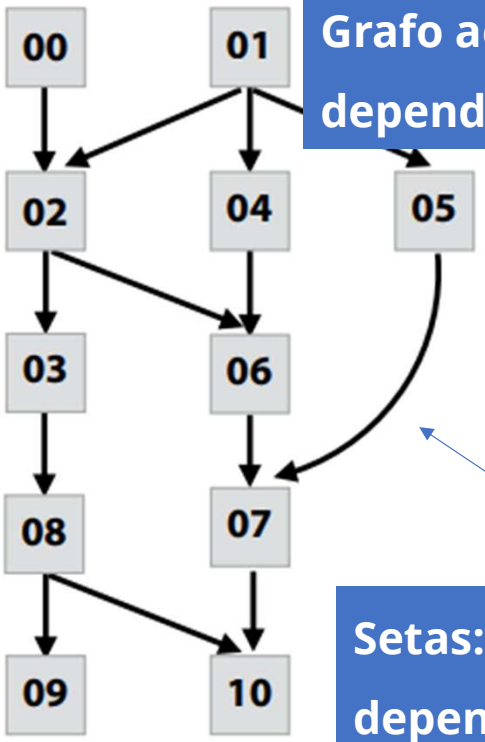
**Program (sequence of instructions)**

| PC | Instruction | |
|---|---|---|
| 00 | a = 2 | |
| 01 | b = 4 | |
| | | *value during execution* |
| 02 | tmp2 = a + b | // 6 |
| 03 | tmp3 = tmp2 + a | // 8 |
| 04 | tmp4 = b + b | // 8 |
| 05 | tmp5 = b * b | // 16 |
| 06 | tmp6 = tmp2 + tmp4 | // 14 |
| 07 | tmp7 = tmp5 + tmp6 | // 30 |
| 08 | if (tmp3 > 7) | |
| 09 | print tmp3 | |
| | else | |
| 10 | print tmp7 | |

Explicação no próximo slide...

## Visualizando o grafo de dependência.
## Identificando os níveis de independência e Dependencia.

**Instruction dependency graph**




Grafo acíclico dirigido o grafo de dependência.

Setas: indicam o fluxo da dependência.

## Outro exemplo, um pouco mais complexo

**O grafo de dependência:** o que ele demonstra?

→O Grafo da Direita, chamado de **Instruction Dependency Graph**, é um Grafo Acíclico Dirigido (DAG) onde cada nó representa uma instrução específica do programa (PC).

As setas, também chamadas de arestas, indicam o fluxo de dependência.

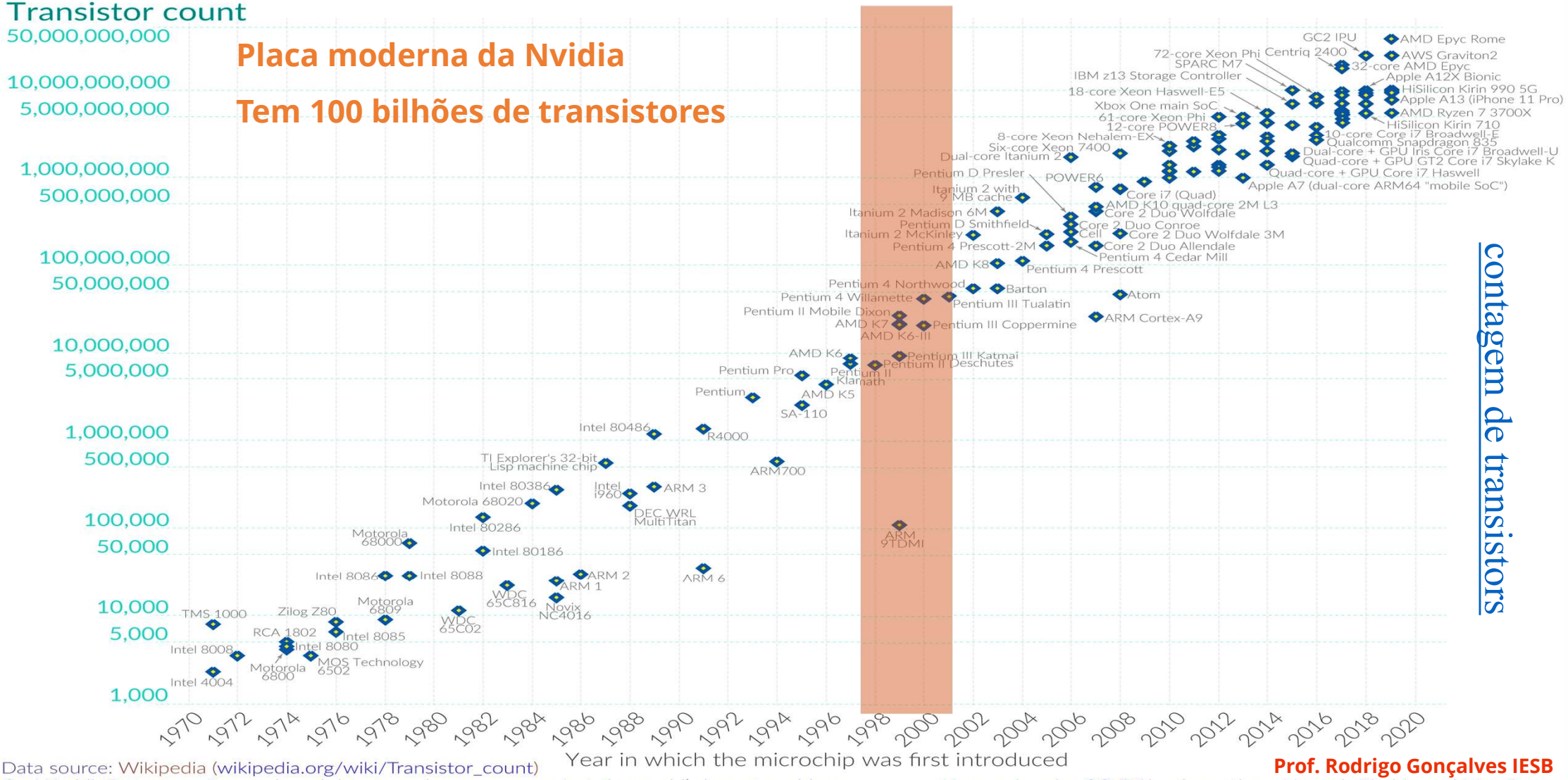
Uma seta apontando do nó A para o nó B significa estritamente:**"A instrução B não pode começar até que a instrução A seja concluída".**

→**Note o nó 03:** Ele só pode ser executado após a conclusão da instrução 02 (que calcula a variável tmp2). Detalhe Importante: Ao analisar a **instrução 03 (tmp3 = tmp2 + a),** percebe-se que não existe uma seta saindo da instrução 00 (onde a foi definido) até a instrução 03. Isso ocorre porque o processador superescalar já tem o valor de a armazenado em um registrador. O grafo sofre uma redução transitiva, omitindo setas redundantes para focar apenas nas dependências imediatas que realmente geram tempo **de espera.**

# Moore's Law: The number of transistors on microchips doubles every two years

Our World in Data

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years.
This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.




Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)
OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

Prof. Rodrigo Gonçalves IESB

# The "power wall"

**A tradução da energia é CALOR, e CALOR precisa de resfriamento**

**Power consumed by a transistor:**

**Dynamic power $\propto$ capacitive load $\times$ voltage$^2$ $\times$ frequency**

**Static power: transistors burn power even when inactive due to leakage**



**High power = high heat**

**Power is a critical design constraint in modern processors**

| | TDP |
|---|---|
| Apple M1 laptop: | 13W |
| Intel Core i9 10900K (in desktop CPU): | 95W |
| NVIDIA RTX 4090 GPU | 450W |
| Mobile phone processor | 1/2 - 2W |
| World's fastest supercomputer | megawatts |
| Standard microwave oven | 900W |



**Explicação próximos slides...**

Source: Intel, NVIDIA, Wikipedia, Top500.org

## → A Equação da Potência Dinâmica.

O consumo de energia de um transistor quando ele está ativamente mudando de estado (processando dados) é representado pela relação

$$\text{Dynamic power} \propto \text{capacitive load} \times \text{voltage}^2 \times \text{frequency}$$

→ **Capacitive load):** A carga capacitiva, que depende do design e do tamanho físico dos transistores na arquitetura do chip.

→ **Voltage:** A tensão (voltagem) operacional. O fator crítico aqui é que a voltagem está ao quadrado.

→ **Frequency:** A frequência do processador (o famoso "clock", medido em GHz)

→ **O gargalo:** Historicamente, para aumentar a frequência F de um processador para que ele execute instruções mais rápido, os engenheiros precisavam aumentar a voltagem V para manter a estabilidade do sinal. Como a voltagem **cresce de forma quadrática na equação**, aumentar um pouco a velocidade do chip causava um aumento exponencial no consumo de energia.

→ **Leakage:** Potência estática

→**Static Power.** Com a miniaturização extrema dos componentes , ou seja, chips na casa dos **poucos nanômetros**, o isolamento dos transistores deixa de ser perfeito.

→ Mesmo quando o transistor está inativo ,desligado, ocorre um vazamento contínuo de corrente.

→ Isso significa que o **chip queima energia e gera calor só por estar ligado na energia**.

→ **Calor e o Limite Físico**→ High power = high heat. Toda essa energia elétrica consumida se converte em calor. A foto do dissipador de calor robusto no slide ilustra a consequência: resfriar esses componentes virou um problema monumental. Se a indústria continuasse escalando a frequência e a voltagem como fazia até meados dos anos 2000, os processadores gerariam tanto calor por milímetro quadrado quanto um reator nuclear, derretendo o silício

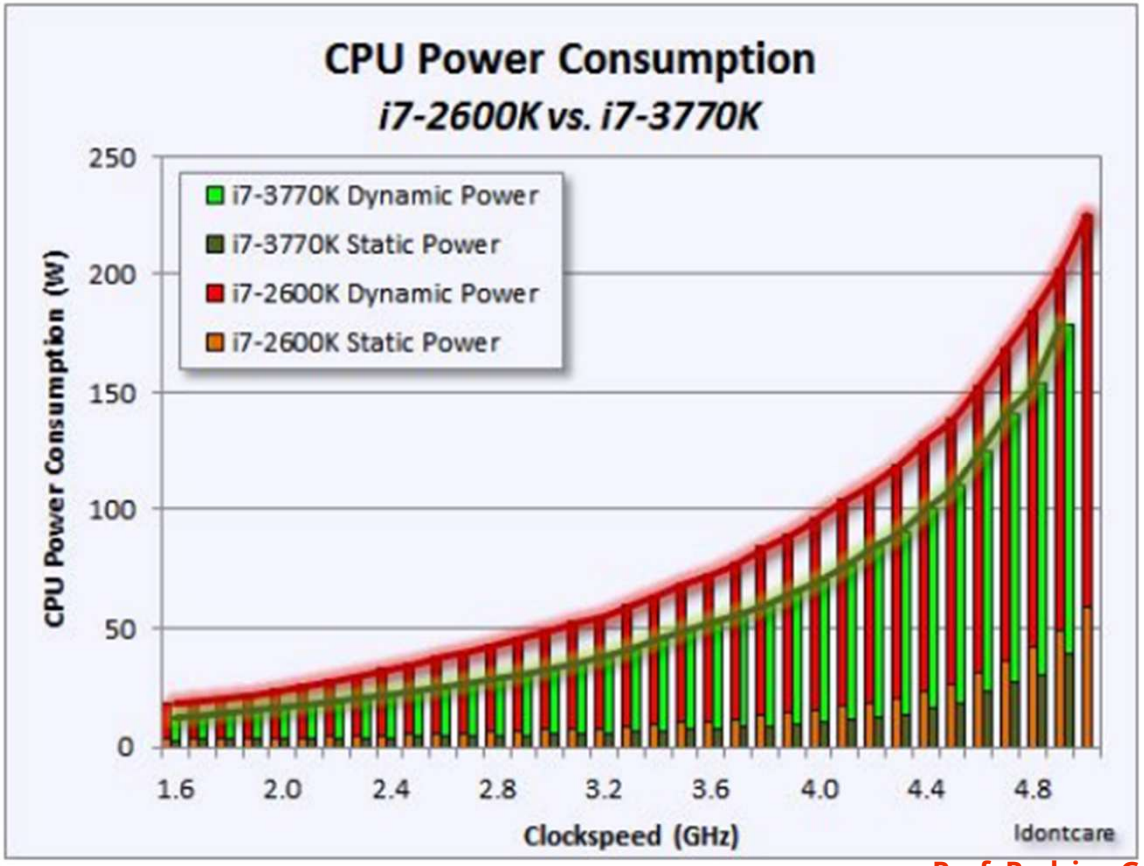
# Power draw as a function of clock frequency

**Dynamic power $\propto$ capacitive load $\times$ voltage$^2$ $\times$ frequency**

**Static power: transistors burn power even when inactive due to leakage**

**Maximum allowed frequency determined by processor's core voltage**

**Energia é altamente correlacionada com o aumento da frequência!**



# Single-core performance scaling

**The rate of single-instruction stream performance scaling has decreased (almost to zero)**

1. **Frequency scaling limited by power**
2. **ILP scaling tapped out**

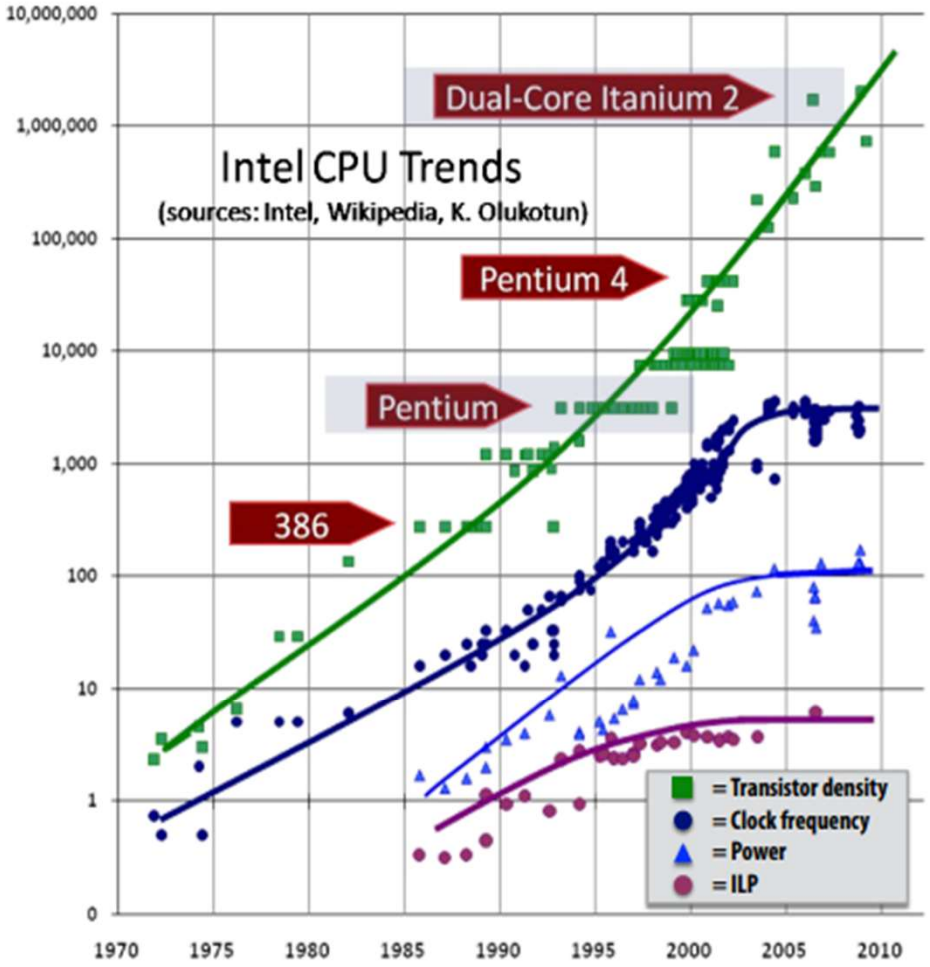
**Atualmente o os especialistas estão fazendo?**

**Architects are now building faster processors by adding more execution units that run in parallel**

**(Or units that are specialized for a specific task: like graphics, or audio/video playback)**

**Software must be written to be parallel to see performance gains. No more free lunch for software developers!**

**Vamos interpretar o gráfico**



**Nvidia AD102 GPU:** contém 76 bilhões de transistores.

**chip utilizado**.

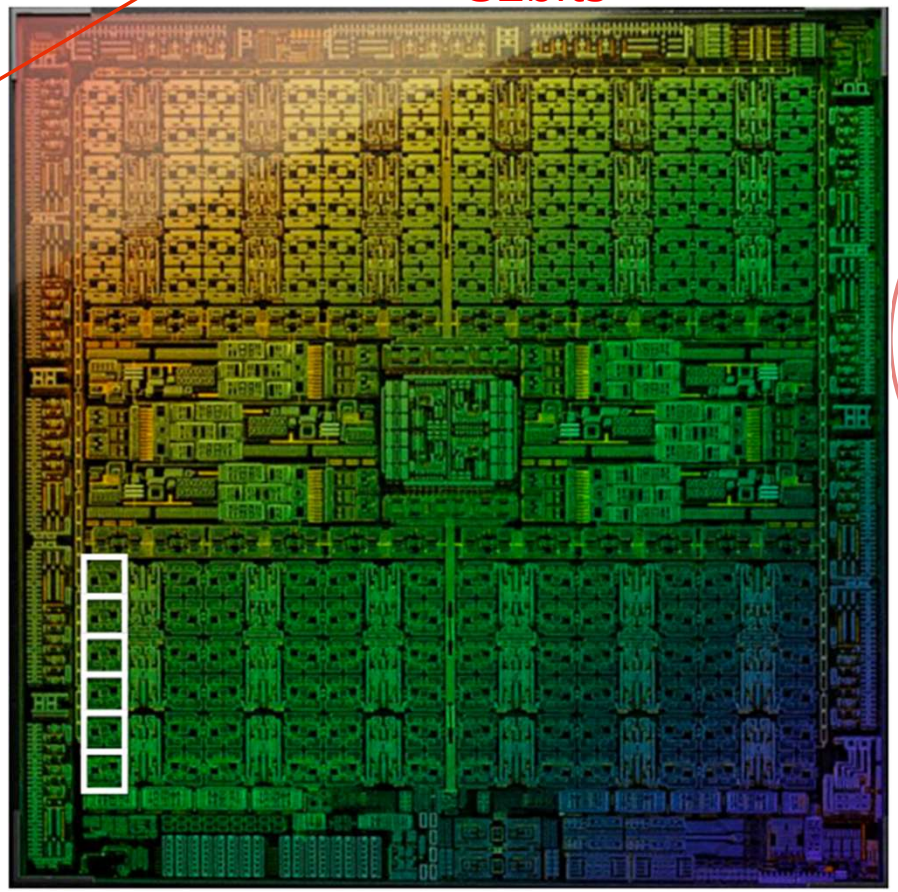
**NVIDIA AD102 GPU**

**GeForce RTX 4090 (2022)**

**76 billion transistors**

**18,432 fp32 multipliers organized in 144 processing blocks (called SMs)**

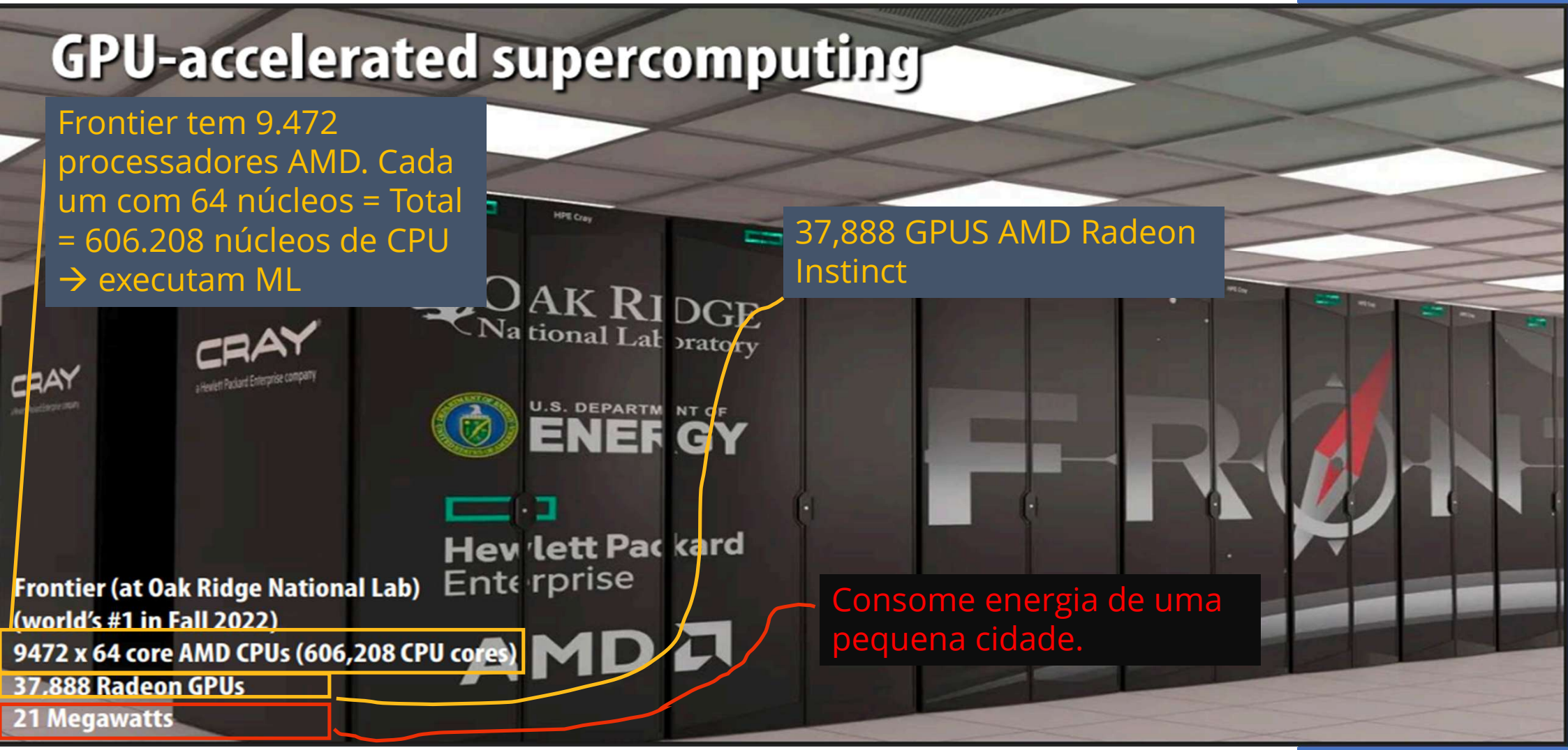
qtde unidades capazes de fazerem multiplicações em ponto flutuantes de 32bits



Esses multiplicadores estão agrupados em 144 blocos de processamento, também conhecidos como Streaming Multiprocessors (SMs)

IESB Centro Universitário A Teoria e a Prática Juntas

## Super Computer

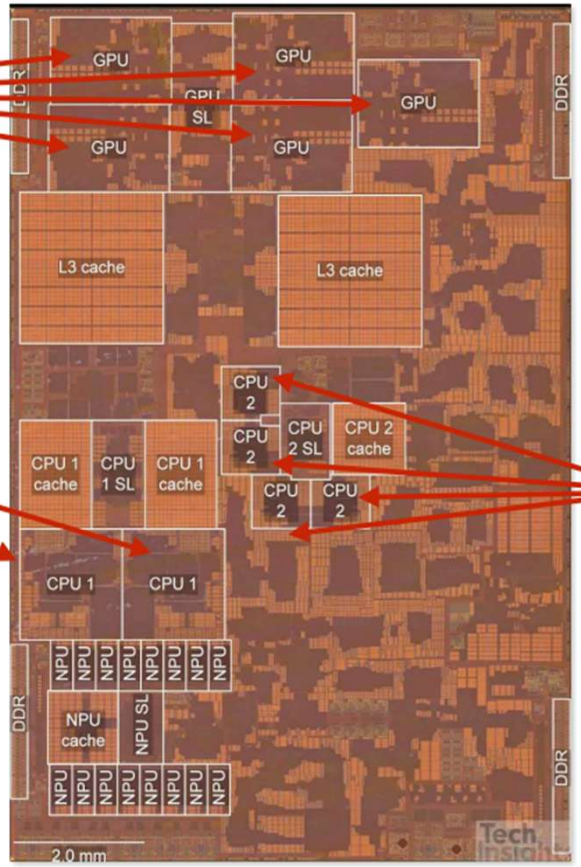


## Consumo de energia também limita a construção de smartphones.

## Arquitetura do Iphone 13,14

# Mobile parallel processing

**Power constraints also heavily influence the design of mobile systems**




5 GPU blocks

2 "big" CPU cores

4 "small" CPU cores

**Apple A15 Bionic (in iPhone 13, 14)**

**15 billion transistors**
**6-core CPU**
**Multi-core GPU**

Image Credit: TechInsights Inc.

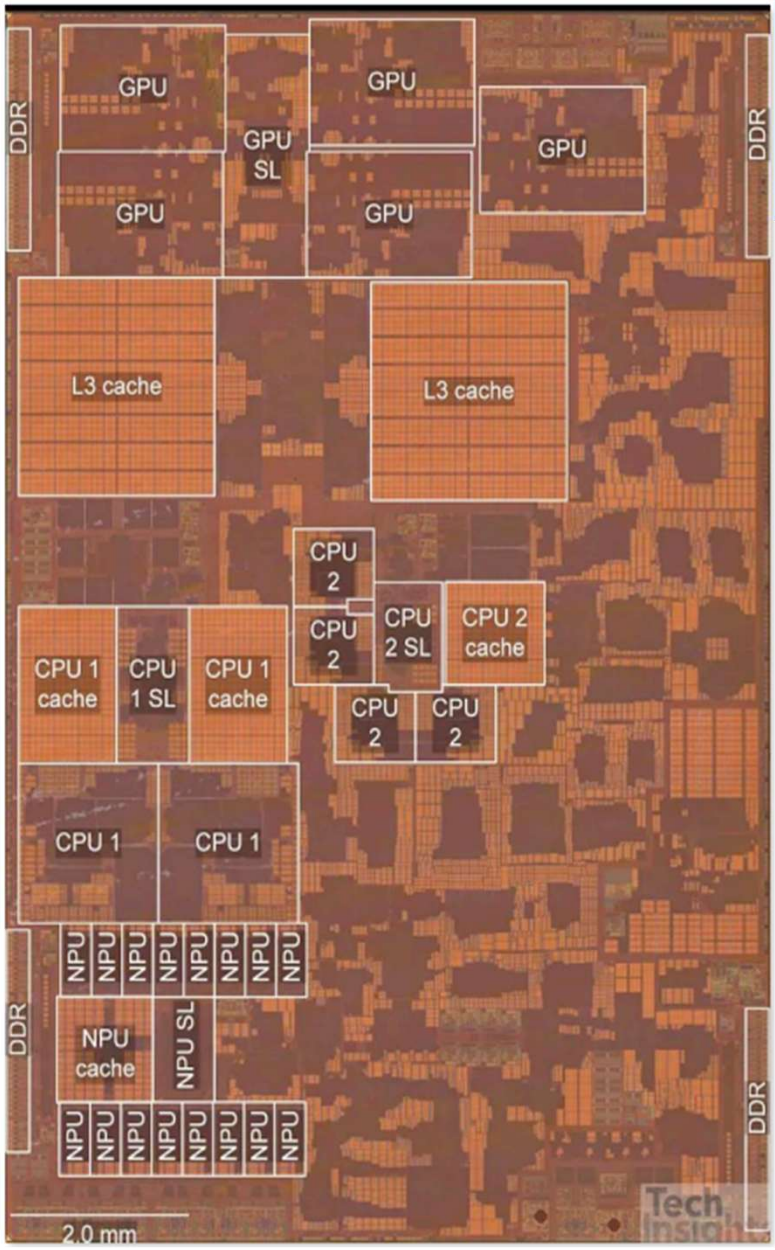
Prof. Rodrigo Gonçalves IESB

# Como resolver o problema do consumo de energia?




**R. Com a utilização de Processadores otimizados, para tarefas específicas!**

Prof. Rodrigo Gonçalves IESB




**Com o processamento especializado, para realização de tarefas específicas. Além da CPU principal. → foco em EFICIÊNCIA**



**Arquitetura –iphone 14**
**Apple A15 Bionic.**

IESB
Centro Universitário
A Teoria e a Prática Juntas

→ CPU: núcleos big e small → tarefas gerais
→ GPU: Graphics Processing Unit → renderização de imagens, vídeos, jogos, gráficos.

→ NPU / Neural Engine: → IA e ML

# Por quê processadores otimizados consomem menos energia?

**Por que eles são projetados apenas para executar um conjunto limitado de tarefas, que ao contrário de uma CPU genérica, necessita estar preparada para executar TODO o TIPO de tarefa, com qualquer tipo de instrução**

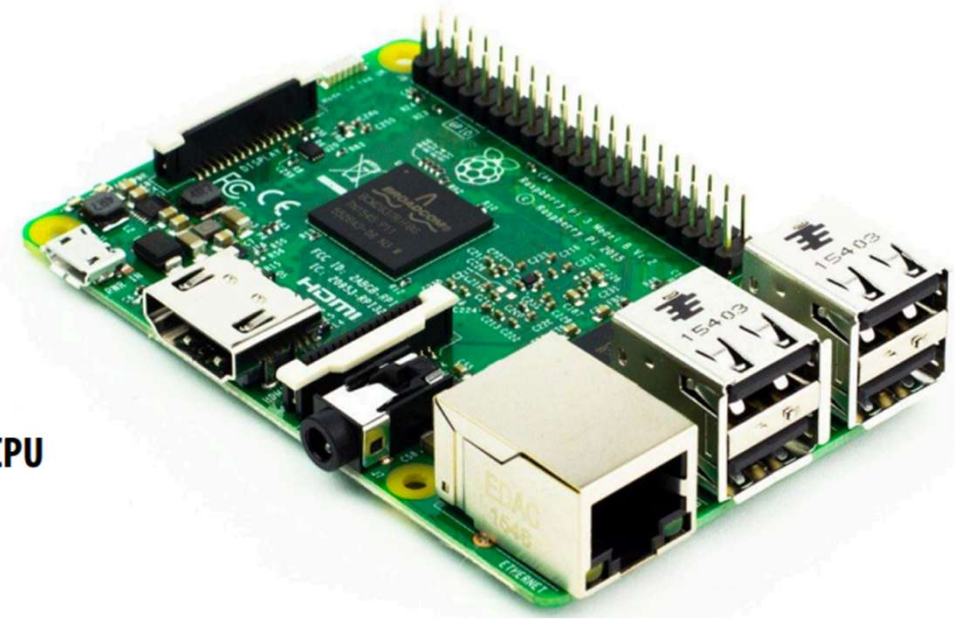



**Circuitos otimizados:** tem caminhos de dados, unidades de execução e lógica de controle para uma tarefa específica **= menos transistores ativos = menos capacitância elétrica = menos energia por operação**

Prof. Rodrigo Gonçalves IESB

# Consumo de energia é uma restrição para a construção de dispositivos móveis

## Mobile parallel processing



**Raspberry Pi 3**

**Quad-core ARM A53 CPU**

**4 núcleos com paralelismo**

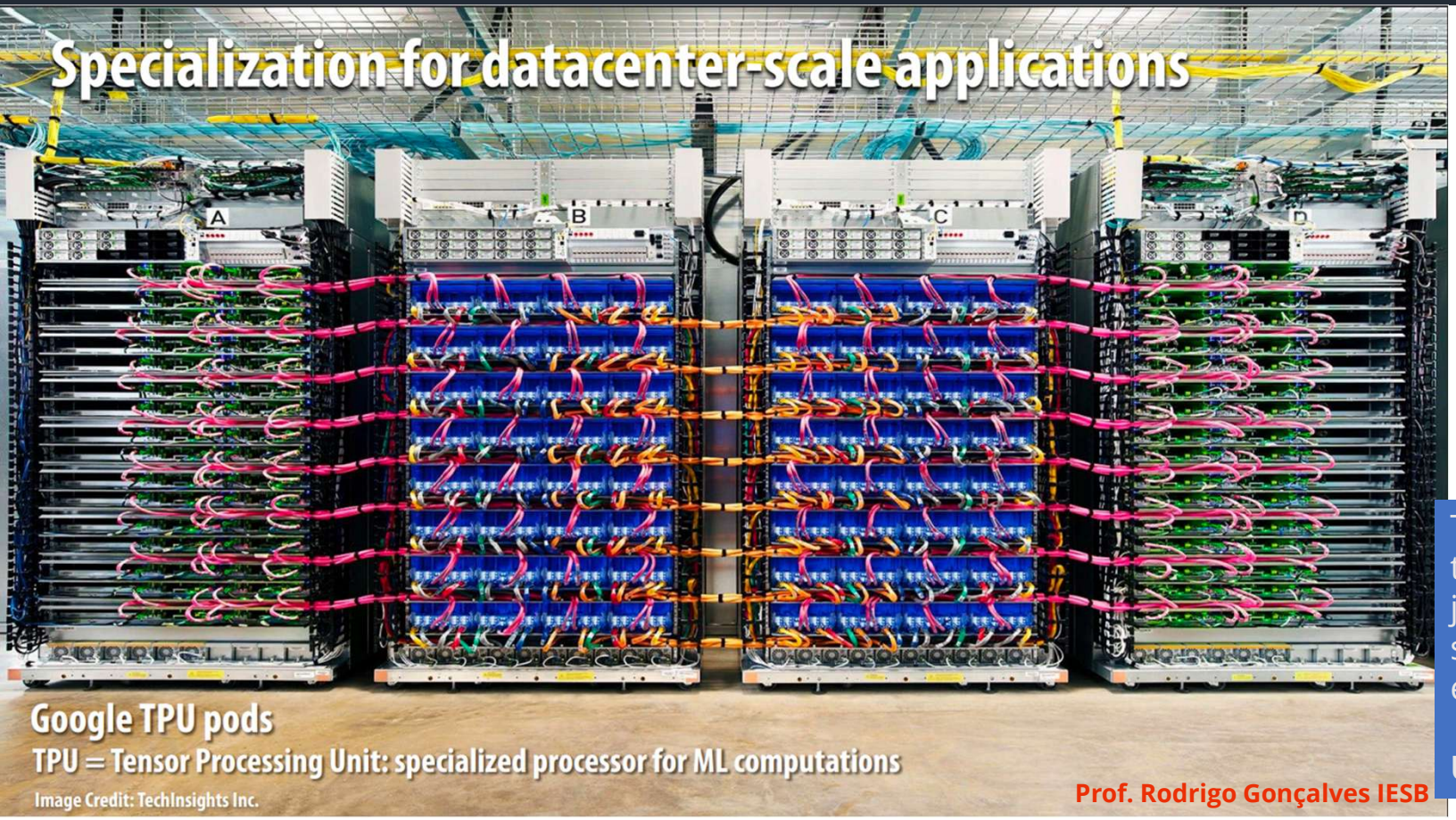
**Baixo custo**

**Ideal para construção de clusters**

**Baixo custo de energia**

Prof. Rodrigo Gonçalves IESB

**Para processamento em larga escala (data centers) como:**
**→ Google;**
**→ Amazon;**
**→ Microsoft. → chips personalizados, como o TPU.**




IESB
Centro Universitário
A Teoria e a Prática Juntas

TPU Pods = conjunto de várias tpus interligadas que trabalham juntas como um supercomputador especializado em ML.

**Utilizado para treinar o GEMINI.**



**Conseguem mais operações por segundo com menor consumo de energia do que uma CPU**

# Vamos conhecer data center da google?

Google Data Center online → escolher legenda em português

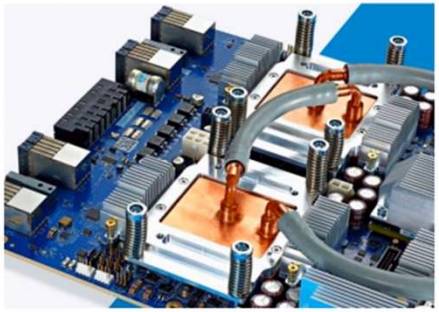
Google Data center offline em inglês
Na pasta da aula.

Prof. Rodrigo Gonçalves IESB

# Outros chips especialistas

**Deep Neural Networks**

## Specialized hardware to accelerate DNN inference/training



Google TPU3



GraphCore IPU




Apple Neural Engine

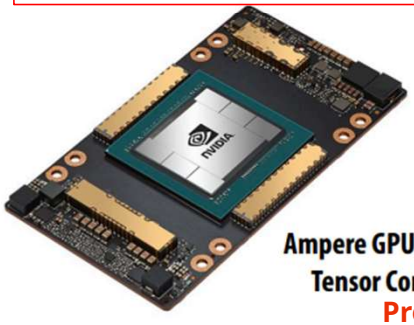
Intelligence Processing Unit




Intel Deep Learning
Inference Accelerator

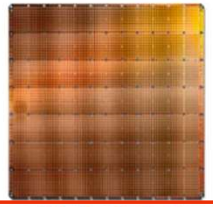



SambaNova
Cardinal SN10



Ampere GPU with
Tensor Cores

Chip gigante, tamanho
De uma bolacha, de
silício.



Cerebras Wafer Scale Engine

IESB
Centro Universitário
A Teoria e a Prática Juntas

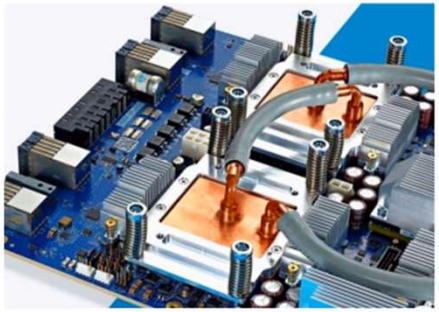
Cada player, tem sua tecnologia

Prof. Rodrigo Gonçalves IESB

# Outros chips especialistas

**Deep Neural Networks**

## Specialized hardware to accelerate DNN inference/training



Google TPU3



GraphCore IPU




Apple Neural Engine

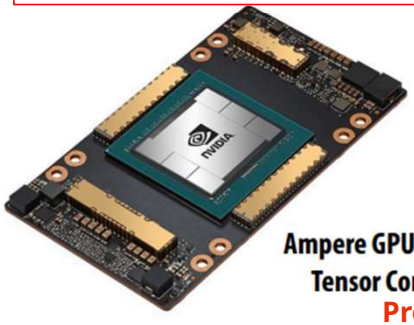
Intelligence Processing Unit




Intel Deep Learning
Inference Accelerator

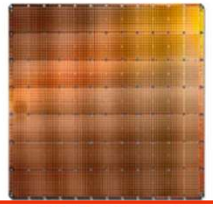



SambaNova
Cardinal SN10



Ampere GPU with
Tensor Cores

Chip gigante, tamanho
De uma bolacha, de
silício.



Cerebras Wafer Scale Engine

IESB
Centro Universitário
A Teoria e a Prática Juntas

Cada player, tem sua tecnologia

Prof. Rodrigo Gonçalves IESB

# Arquitetura Samsung S24




Qualcomm Technologies → San Diego, California

Prof. Rodrigo Gonçalves IESB

# Qualcomm Snapdragon

**Esperar resposta da fabricante e colocar aqui.**

**Prof. Rodrigo Gonçalves IESB**

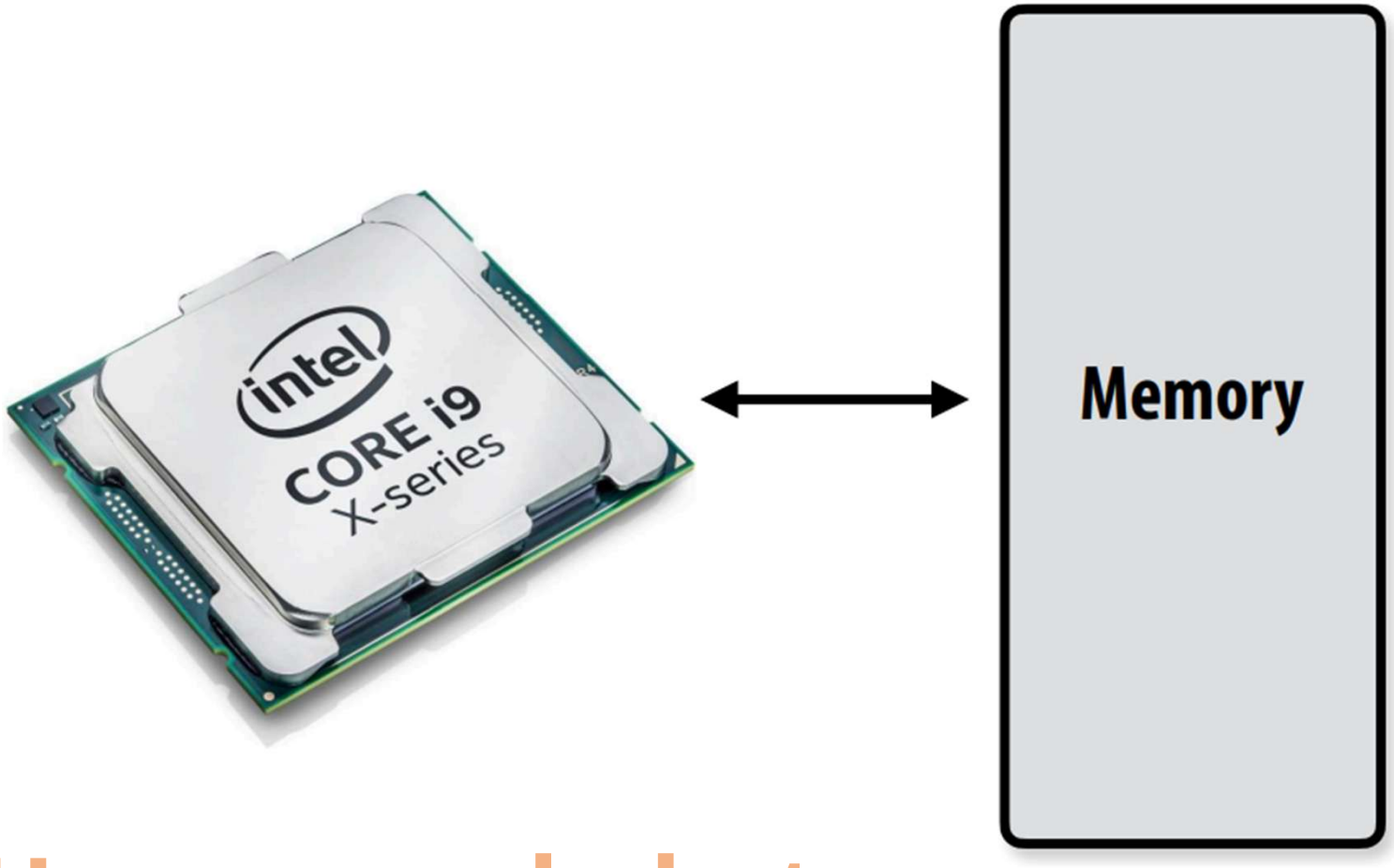
# Um dos pilares para processamento eficiente, é o acesso a memória.

# Vamos entendê-las melhor




Prof. Rodrigo Gonçalves IESB

# O que é memória?




## Um array de bytes.

Prof. Rodrigo Gonçalves IESB

| Address | Value |
|---|---|
| 0x0 | 16 |
| 0x1 | 255 |
| 0x2 | 14 |
| 0x3 | 0 |
| 0x4 | 0 |
| 0x5 | 0 |
| 0x6 | 6 |
| 0x7 | 0 |
| 0x8 | 32 |
| 0x9 | 48 |
| 0xA | 255 |
| 0xB | 255 |
| 0xC | 255 |
| 0xD | 0 |
| 0xE | 0 |
| 0xF | 0 |
| 0x10 | 128 |
| ⋮ | ⋮ |
| 0x1F | 0 |

**Cada byte é identificado pelo seu endereço na memória: o endereço é a posição do array. (hexadecimal)**

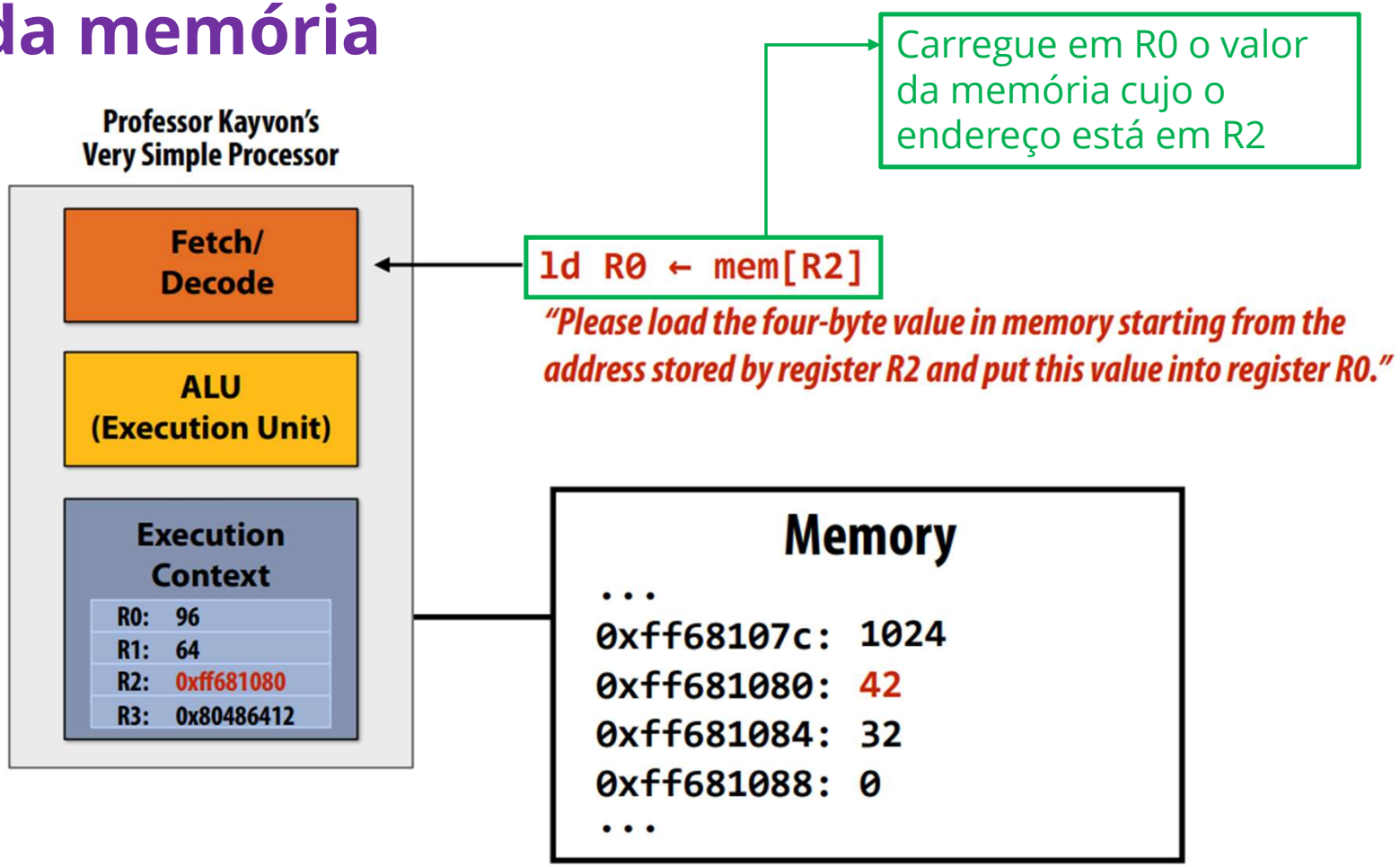
→ **assume-se então,** que a memória é um byte endereçado.

O byte: salvo no endereço 0x8 tem o valor 32.

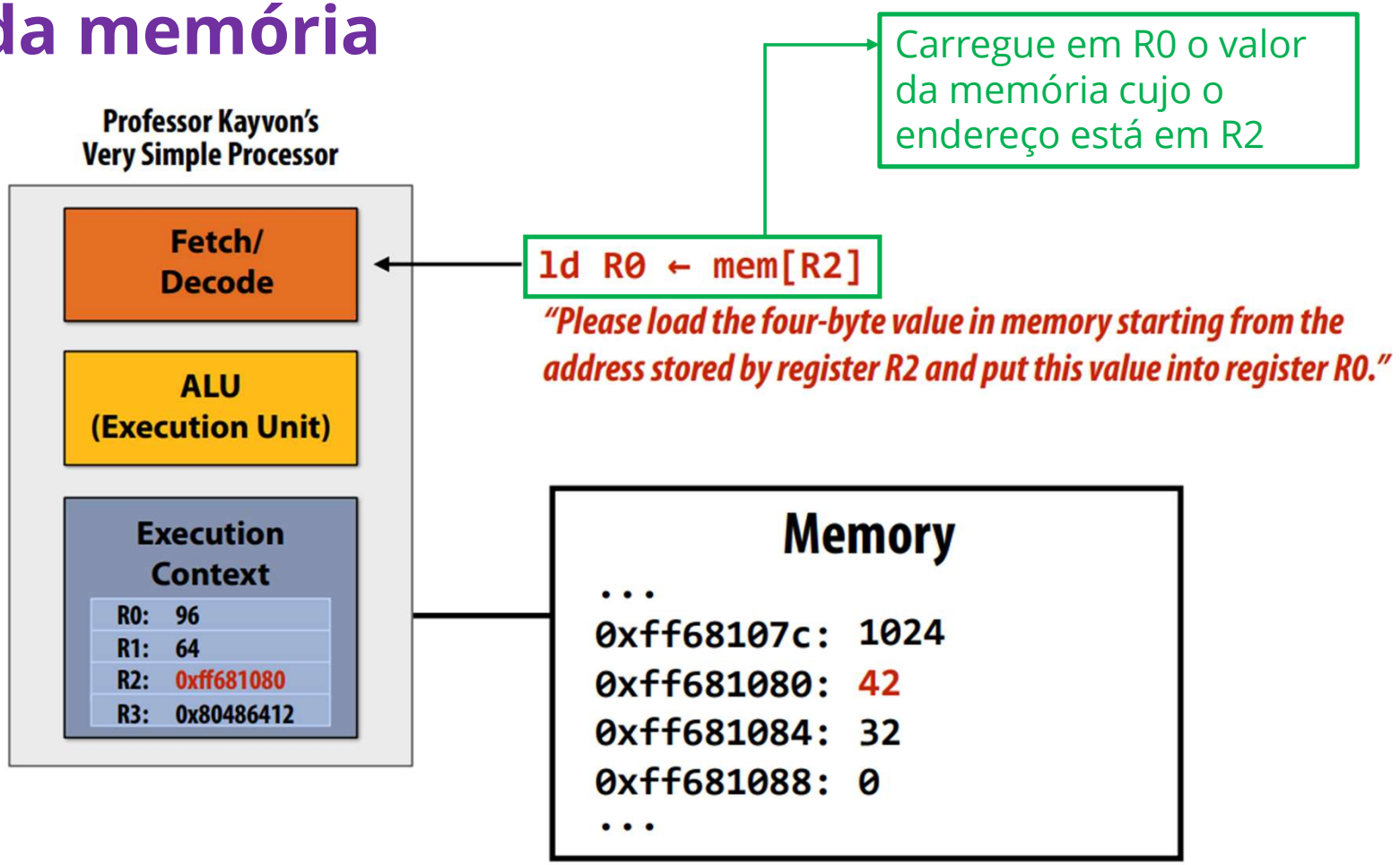
O byte: salvo no endereço 0x10 (16) tem o valor 128.



# Carregando no processador um valor da memória



# Carregando no processador um valor da memória

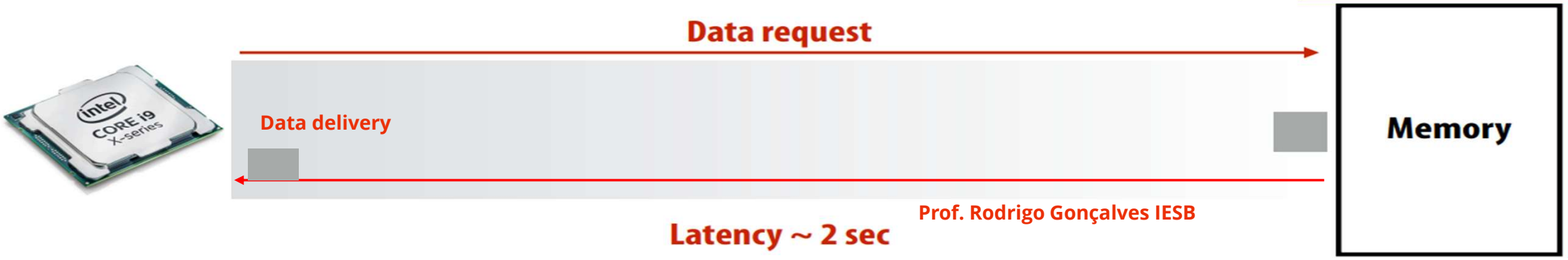



Continua...

# Carregando no processador, um valor da memória

## Vamos falar sobre latência de acesso à memória:

**Latência:** é tempo total gasto para obter o dado da memória. Desde a requisição até a entrega do dado no processador.



# **Stalls =** parada ou travamento temporário do processador.

→ **Além,** da latência, o processador entra em "stall" quando a próxima instrução não pode ser executada imediatamente porque depende de um resultado que ainda não está pronto.

→ **Causas comuns:**

a) **Acesso a memória DRAM** pode levar a centenas de ciclos para chegar.
b) **Dependência de dados:** uma instrução depende do resultado anterior.
c) **Sincronização da programação paralela:** esperar outro thread ou processo liberar um recurso.

→ **Consequências do stall para desempenho:**

a) O processador não aproveita todo o potencial de execução.
b) Frequentemente, o gargalo de desempenho não é velocidade de execução, mas frequência e duração desses stalls.

# Stalls → exemplo

```
ld r0 mem[r2]
ld r1 mem[r3]
add r0, r0, r1
```

carregue em R0 o valor da memória cujo o endereço está em r2.

carregue em R1 o valor da memória cujo o endereço está em r3.

Adição não pode ser executada até mem[r2] e mem[r3] serem carregados na memória




A memória DRAM (Dymamic Random Access Memory) é lenta. ~ 100 ciclos de clock!

Como resolver problemas de STALLS ?

**Memórias cache!**

# Memória Cache

**Cache** é uma implementação de hardware que não impacta na saída ou resultado de um programa, ele é utilizado apenas com o objetivo de melhorar a performance

**Cache** é um chip que armazena uma cópia de subconjunto de valores que estão na memória.

**Cache** guarda cópias da memória principal que foram usadas recentemente ou que o processador usará em breve.

**Cache** fica alocada dentro do processador, atualmente.



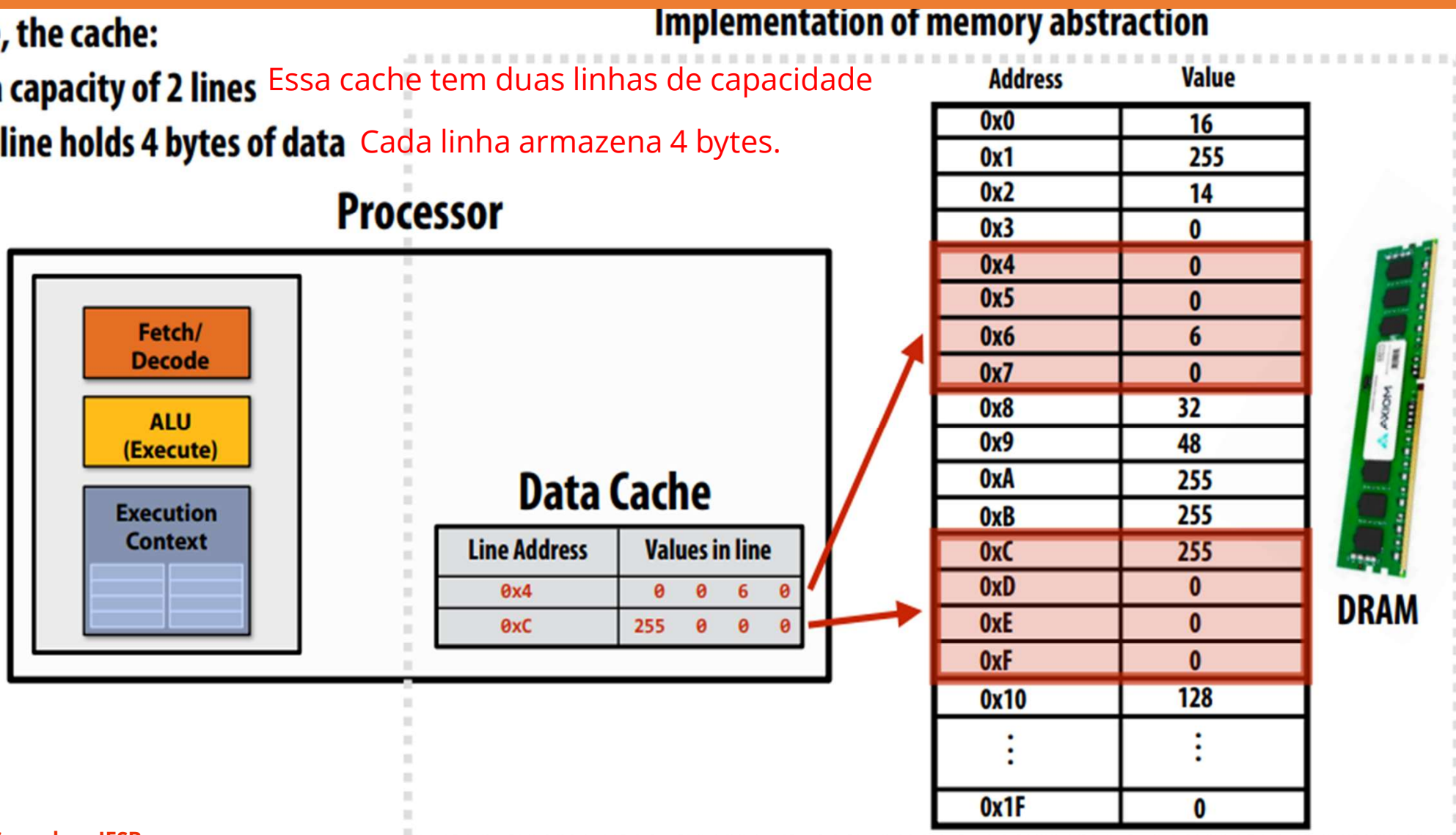

→**Se um endereço** é armazenado **"em cache"** o processador pode carregar / armazenar esse endereço muito mais rapidamente do que os dados que estão apenas na memória RAM.

**No próximo slide, vamos ver um exemplo de como a memória cache funciona.**

# Memória Cache

In the figure, the cache:

- Has a capacity of 2 lines Essa cache tem duas linhas de capacidade
- Each line holds 4 bytes of data Cada linha armazena 4 bytes.



# Memória Cache

## Cache example 1

Array of 16 bytes in memory DRAM

| | Address | Value |
|---|---|---|
| Line 0x0 | 0x0 | 16 |
| | 0x1 | 255 |
| | 0x2 | 14 |
| | 0x3 | 0 |
| Line 0x4 | 0x4 | 0 |
| | 0x5 | 0 |
| | 0x6 | 6 |
| | 0x7 | 0 |
| Line 0x8 | 0x8 | 32 |
| | 0x9 | 48 |
| | 0xA | 255 |
| | 0xB | 255 |
| Line 0xC | 0xC | 255 |
| | 0xD | 0 |
| | 0xE | 0 |
| | 0xF | 0 |

Assume:

Total cache capacity of 8 bytes

Cache with 4-byte cache lines
(So 2 lines fit in cache)

Least recently used (LRU) replacement policy

Substitui a linha menos recentemente usada

time

| Address accessed | Cache action | Cache state (after load is complete) |
|---|---|---|
| 0x0 | "cold miss", load 0x0 | 0x0 ●●●● |
| 0x1 | hit | 0x0 ●●●● |
| 0x2 | hit | 0x0 ●●●● |
| 0x3 | hit | 0x0 ●●●● |
| 0x2 | hit | 0x0 ●●●● |
| 0x1 | hit | 0x0 ●●●● |
| 0x4 | "cold miss", load 0x4 | 0x0 ●●●● 0x4 ●●●● |
| 0x1 | hit | 0x0 ●●●● 0x4 ●●●● |

não está no cache, carregar

O que acontece: o registro 0x0 não está no cache, logo o cache carrega a linha que contém 0x0 → e cobre os endereços de 4 bytes por linha: 0x0, 0x1, 0x2, 0x3. Os valores que já estão no cache, dessa leitura, ele dá um **hit. O mesmo** acontece com o 0x4, e assim por diante.

# Memória Cache

## Cache example 1

Array of 16 bytes in memory DRAM

| | Address | Value |
|---|---|---|
| Line 0x0 | 0x0 | 16 |
| | 0x1 | 255 |
| | 0x2 | 14 |
| | 0x3 | 0 |
| Line 0x4 | 0x4 | 0 |
| | 0x5 | 0 |
| | 0x6 | 6 |
| | 0x7 | 0 |
| Line 0x8 | 0x8 | 32 |
| | 0x9 | 48 |
| | 0xA | 255 |
| | 0xB | 255 |
| Line 0xC | 0xC | 255 |
| | 0xD | 0 |
| | 0xE | 0 |
| | 0xF | 0 |

Assume:

Total cache capacity of 8 bytes

Cache with 4-byte cache lines
(So 2 lines fit in cache)

Least recently used (LRU) replacement policy

Substitui a linha menos recentemente usada

| Address accessed | Cache action | Cache state (after load is complete) |
|---|---|---|
| 0x0 | "cold miss", load 0x0 | 0x0 ●●●● |
| 0x1 | hit | 0x0 ●●●● |
| 0x2 | hit | 0x0 ●●●● |
| 0x3 | hit | 0x0 ●●●● |
| 0x2 | hit | 0x0 ●●●● |
| 0x1 | hit | 0x0 ●●●● |
| 0x4 | "cold miss", load 0x4 | 0x0 ●●●● 0x4 ●●●● |
| 0x1 | hit | 0x0 ●●●● 0x4 ●●●● |

time

não está no cache, carregar

**Há 2 tipos de cache:** localidade (espacial ou temporal.

→ **Espacial:** quando um end. de mem. é acessado, é provável que logo depois, os demais endereços apresentam hits.
→ **Temporal:** quando você acessa um end. de mem. É provável que o acesse novamente em breve. Se o dado continuar no cache quando precisar dele outra vez, será um hit.

# Memória Cache – Exemplo 2

## Cache example 2

Array of 16 bytes in memory

| | Address | Value |
|---|---|---|
| Line 0x0 | 0x0 | 16 |
| | 0x1 | 255 |
| | 0x2 | 14 |
| | 0x3 | 0 |
| Line 0x4 | 0x4 | 0 |
| | 0x5 | 0 |
| | 0x6 | 6 |
| | 0x7 | 0 |
| Line 0x8 | 0x8 | 32 |
| | 0x9 | 48 |
| | 0xA | 255 |
| | 0xB | 255 |
| Line 0xC | 0xC | 255 |
| | 0xD | 0 |
| | 0xE | 0 |
| | 0xF | 0 |

Assume:

Total cache capacity of 8 bytes

Cache with 4-byte cache lines
(So 2 lines fit in cache)

Least recently used (LRU)
replacement policy

LRU escolhe o bloco menos utilizado recentemente para desalojar

time

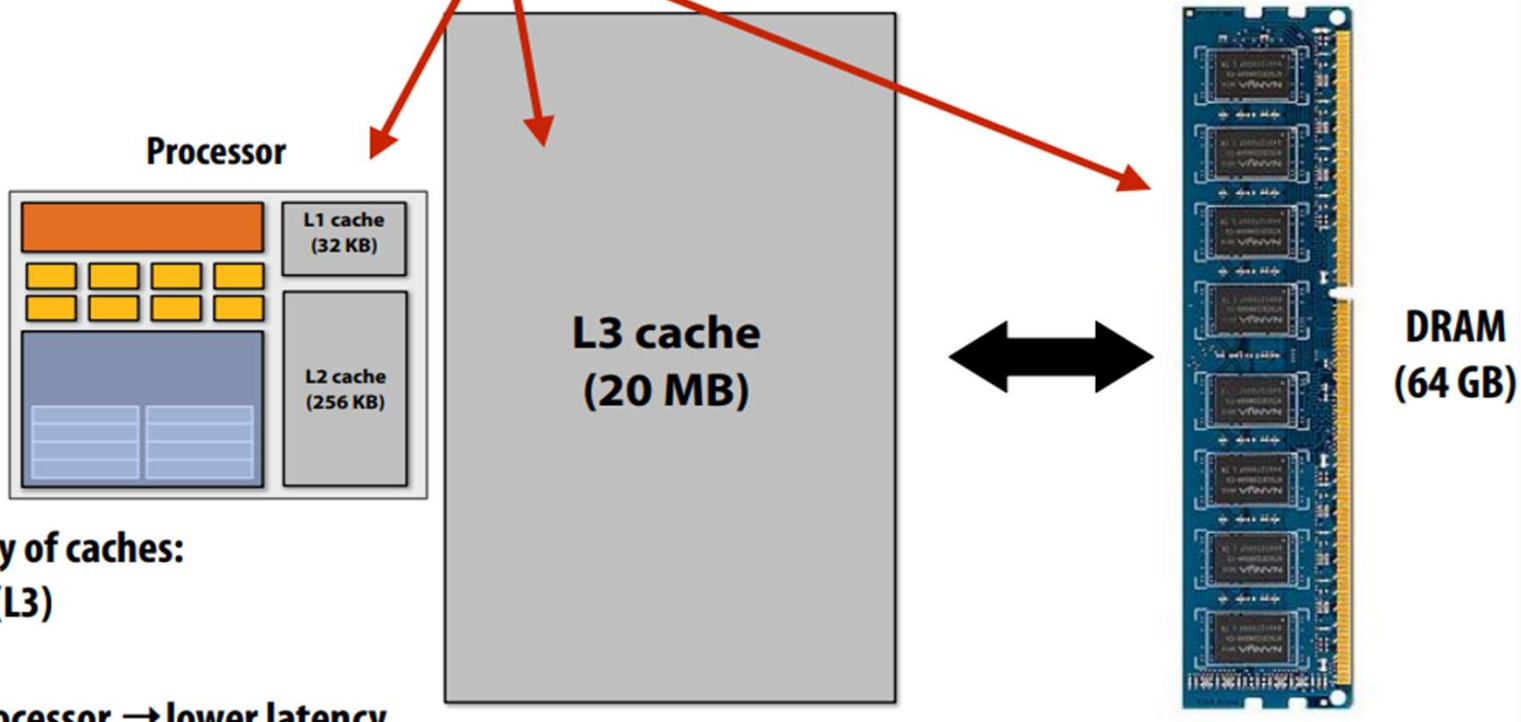
evict = desalojar/ remover

| Address accessed | Cache action | Cache state (after load is complete) |
|---|---|---|
| 0x0 | "cold miss", load 0x0 | 0x0 |
| 0x1 | hit | 0x0 |
| 0x2 | hit | 0x0 |
| 0x3 | hit | 0x0 |
| 0x4 | "cold miss", load 0x4 | 0x0 0x4 |
| 0x5 | hit | 0x0 0x4 |
| 0x6 | hit | 0x0 0x4 |
| 0x7 | hit | 0x0 0x4 |
| 0x8 | "cold miss", load 0x8 (evict 0x0) | 0x8 0x4 |
| 0x9 | hit | 0x8 0x4 |
| 0xA | hit | 0x8 0x4 |
| 0xB | hit | 0x8 0x4 |
| 0xC | "cold miss", load 0xC (evict 0x4) | 0x8 0xC |
| 0xD | hit | 0x8 0xC |
| 0xE | hit | 0x8 0xC |
| 0xF | hit | 0x8 0xC |
| 0x0 | "capacity miss", load 0x0 (evict 0x8) | 0x0 0xC |

# Memória Cache → Implementação complexa

## The implementation of the linear memory address space abstraction on a modern computer is complex

The instruction "load the value stored at address X into register R0" might involve a complex sequence of operations by multiple data caches and access to DRAM

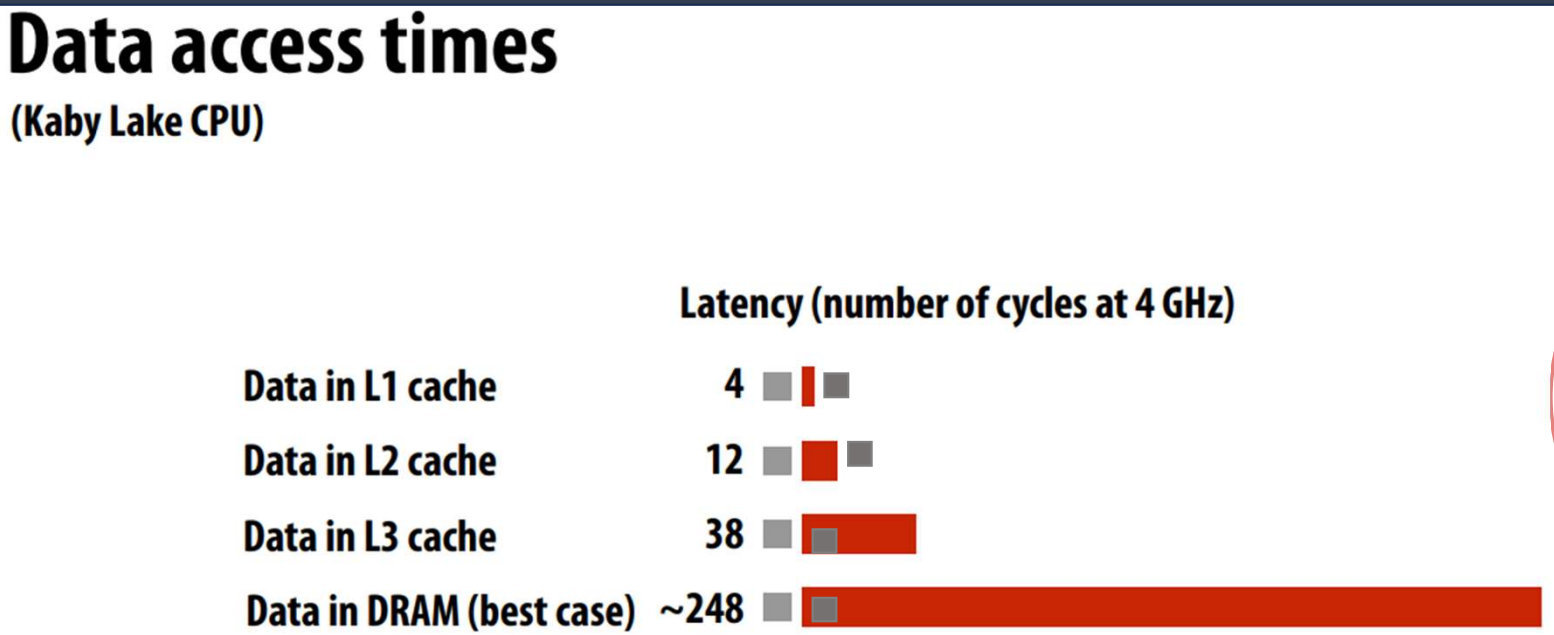



Common organization: hierarchy of caches:
Level 1 (L1), level 2 (L2), level 3 (L3)

Smaller capacity caches near processor → lower latency

Larger capacity caches farther away → larger latency

# Memória cache  tempo estimado de acesso a dado.



## Memória Cache, portanto...

# Memória cache reduz o stall e a latência de acesso a memória.



# Exercícios

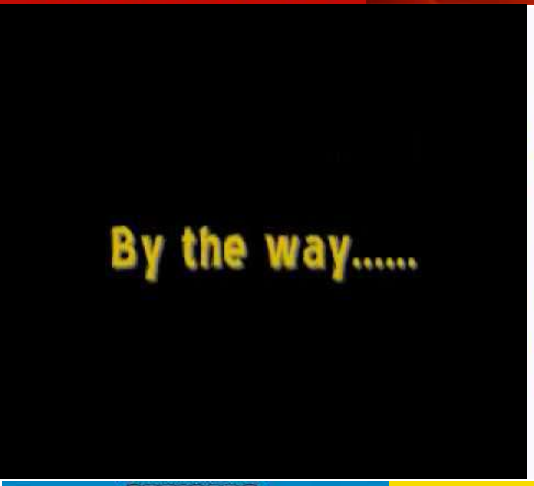



THANKS
FOLKS !!
AND
HOPE YOU
ENJOYED


That's all Folks!


Thank You!


Obrigado
Thank You
Danke
Merci
Grazie
Spasibo
Arigato
Kiitos
Chokrane
Multumesc
Matondo
Mochchakkeram
Dankon
Maake
Welalin
Kia Ora
Vinaka
Mamana
Terma Kasih
Dank Je
Cam on ban
Juspaxar
Nirringrazzjak
Asante
Salamat
Matur Nuwun
Raibh Maith Agat
Ua Tsaug Rau Koj

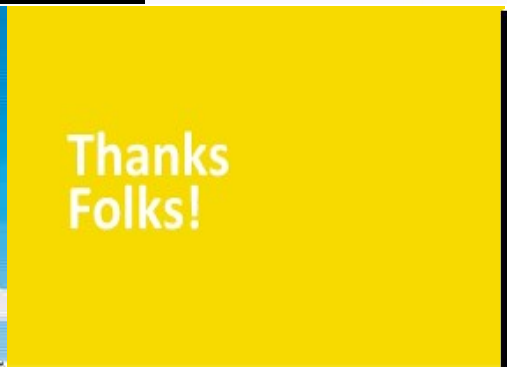

By the way.....


OBRIGADO!


THANK
YOU
GRACIAS
ARIGATO
SHUKURIA
BOLZÏN
MERCI
BÏYAN
DANKSCHEEN
TASHAKKUR ATU
SUKSAMA
MEHRBANI
GRAZIE
JUSPAXAR


OBRIGADO


OBRIGADO
PELA ATENÇÃO


Thanks
Folks!


Thank You!


Thank You !